

Updating KernelCanvas for weightless graph classification

Raul Bezerra Barbosa^{a,*,}, Diego Leonel Cadette Dutra^{a, ID}, Priscila Machado Vieira Lima^{a, ID},
Diego Carvalho^{b, ID}, Felipe Maia Galvão França^{c, ID}

^a Federal University of Rio de Janeiro (UFRJ), Brazil

^b Federal Centre for Technological Education of Rio de Janeiro (CEFET/RJ), Brazil

^c Instituto de Telecomunicações (IT), Universidade do Porto, Portugal

ARTICLE INFO

Communicated by Y. Cheng

Keywords:

Graph classification
Weightless neural networks
Neural networks
Pattern recognition
Supervised learning

ABSTRACT

In a landscape where graph neural networks establish themselves as a standard in graph learning, we introduce a contrasting weightless graph learning architecture. We propose a novel representation for shallow graph learning based on weightless neural networks (WNN) and extend current WNN quantization techniques. Additionally, we assess this new architecture across widely adopted datasets. Our findings indicate that employing our representation and WNN models can yield superior results to those attained with extra-large graph neural networks or heavyweight kernel methods while imposing fewer demands on computer resources and memory. Furthermore, our results suggest that other classifiers can leverage the proposed representation for improved performance.

1. Introduction

Our community has seen a recent trend of machine learning models using graphs as input and output. The motivation itself is clear: graphs can easily represent nearly every set relationship model, from molecules to galaxies. Nevertheless, it remains an open problem to learn from them. Several approaches of the so-called graph neural networks (GNN) have arisen [1–3], deriving from the message-passing framework [4] or earlier methods [5,6] with promising results. However, apart from the inner problems of graph neural networks, such as over-squashing, over-smoothing, and under-reaching [7], and their limited expressiveness [2,8], graph neural networks are still deep neural networks. These are known powerful hardware and energy consumers [9,10], therefore expensive and red artificial intelligence models [11] that, at some point, we must avoid. Hence, our inspiration is to take a step back and use Occam's razor principle again, focusing on one central question: how much can we learn from graphs if we use lighter models?

Machine learning on graphs has been the subject of many works such as [12–15] and the vast literature agrees that there are at least three main tasks: node classification, link prediction and graph classification. Graph classification is our focus and has its motivation in various fields, such as biology, chemistry, computer science, and physics [16–19]. It poses itself as an entire graph or graph-level learning task. For example, a simple graph classification application predicts whether a substance, represented as a graph, is carcinogenic.

Our main question is answered here with the adoption of a bag of nodes approach, one of the simplest graph representation models [20], used in traditional shallow learning processes. This naive representation can then be processed and fed into lighter classifiers, using every possible graph attribute. Here, by lighter classifiers, we pinpoint every possible off-the-shelf fast learner. However, we give special attention to the WiSARD [21] model and some of its derivatives, considering their lightweight design [22,23], easiness of implementation [24], explainability of decisions [25,26], and robustness [27]. The WiSARD-based models have been used throughout history, with close attention recently, consistently demonstrating reduced training time and low computational demand. The seaming between graph learning and WNNs is also enabled by the development of new aggregation and quantization techniques that extend the KernelCanvas [28] (KC), originally intended for time-series tasks. Our work establishes by design a simple and very effective framework for graph classification that takes less hardware, less time, and hence less energy without compromising performance. This is corroborated by the experimental results on widely used datasets using an established evaluation procedure. We also present results for other common ground classifiers, such as RandomForests, alongside GNN models for comparison.

The remainder of this text is organized as follows. In Section 2, we describe graph learning strategies and recent works, as well the basics of the WiSARD model and its evolutions. Section 3 depicts our architecture, showing all key aspects of the three-phase model. Next,

* Corresponding author.

E-mail address: rbbarbosa@cos.ufrj.br (R. Bezerra Barbosa).

Section 4 details all of our experiments, framing every aspect of their reproducibility, showing our results, and analyzing them. Section 5 concludes our work by wrapping up our results and elaborating on some thoughts about possible future works.

2. Background

2.1. Graph representation

A graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is denoted by $\mathcal{V}$, a finite set of vertices or nodes, and $\mathcal{E}$, a finite set of edges. An edge $e_{i,j}$ is a pair $= (v_i, v_k) \mid v_i, v_k \in \mathcal{V}$. The most common way to represent a graph is by using the adjacency matrix $\mathbf{A}$, where $A_{i,j}$ represents a boolean value of whether there is an edge between vertices i and j . An undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a graph in which there is no distinction between edges (i, j) and (j, i) . In this case, the adjacency matrix is symmetric. On the other hand, a directed graph is a graph in which there is a distinction between edges (i, j) and (j, i) .

The adjacency matrix is often replaced by a weight (or proximity) matrix $\mathbf{W}$, where $W_{i,j} \in \mathbb{R}$ represents a quantity about the relationship between i and j . Attributed graphs will also have node and edge attributes, which can be represented as additional matrices, leading to a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X}, \mathbf{Y})$, where $\mathbf{X}$ is a $|\mathcal{V}| \times a$ matrix representing a attributes per node, and $\mathbf{Y}$ is a $|\mathcal{E}| \times b$ matrix, consisted of b attributes per edge. Traditional machine learning techniques, such as the classifiers used in our work, rely on a vector representation of a graph, often with handcrafted features. The adjacency or weight matrix itself could be used as such vector representation, but it is highly dependent on node order or labeling. Hence, to overcome this issue, we pinpoint a selection of simple and traditional graph representation methods.

2.1.1. Graph statistics and bag-of-nodes

The bag-of-nodes (BON) strategy is a naive approach to aggregate node-level measures of a graph to compose a graph-level representation [20]. This aggregation itself can be done using histograms or any other method, such as a simple sum or mean. Node centralities play a key role in graph analysis [29]. A node centrality defines how important a node is in a graph or a subgraph of the original graph according to some heuristic or goal [20].

The node degree is one of the most basic and primarily used node-level measures and can also be seen as a node centrality. It is a local node statistic that counts the neighbors of a node, vertices that are connected in the graph to a specific node i . Mathematically, in an undirected graph, the node degree $d(i)$ is defined by the cardinality of $\mathcal{N}(i)$ where

$$\mathcal{N}(i) = \{k \in \mathcal{V} \mid (i, k) \in \mathcal{E}, i \neq k\} \quad (1)$$

. For directed graphs, two node degrees are observed: the in-degree and the out-degree. The in-degree $d_{in}(k)$ is defined by the cardinality of $\mathcal{N}_{in}(i)$ where

$$\mathcal{N}_{in}(i) = \{k \in \mathcal{V} \mid (k, i) \in \mathcal{E}, i \neq k\} \quad (2)$$

. Similarly, the out-degree $d_{out}(k)$ is defined by the cardinality of $\mathcal{N}_{out}(i)$ where

$$\mathcal{N}_{out}(i) = \{k \in \mathcal{V} \mid (i, k) \in \mathcal{E}, i \neq k\} \quad (3)$$

. The node degree has a trivial complexity of $\mathcal{O}(|\mathcal{E}|)$, being a measure that naturally scales well.

One of the most prominent node centrality is the PageRank centrality [30]. Largely used, the PageRank centrality is also famous for being one of the original building blocks of the Google search engine. With α being a damping factor, a constant $\in (0, 1)$, the page-rank pr_i of a node i is defined as follows:

$$pr_i = \alpha \sum_{k \in \mathcal{N}_{in}(i)} \frac{1}{d_{out}(k)} pr_k + (1 - \alpha) \quad (4)$$

Authors in [30] describe it as an algorithm with fast convergence, especially in real networks, which tend to have a very sparse adjacency matrix. Its exact complexity time depends on the algorithm used for its calculation. It is shown in [31] that, for some cases, the time complexity of its calculation using the power method is linear in $|\mathcal{V}|$.

The onion decomposition is both a local and global-level graph descriptor, and it is conceived by gathering more information during the k -core decomposition algorithm of a graph. The k -core decomposition on its own is a core-periphery analysis tool that aims to define a graph's center (or core) of and its periphery (or shell) [32]. A k -core is a maximal subgraph where every node has at least degree k . The k -shell is the set of all nodes that belong to the k -core but not to the $(k + 1)$ -core. A layer is defined in [32] as the number of peeling passes needed to reach a given node. Authors in [32] also define the onion spectrum, the fraction of nodes that belong to a specific layer (and can be plotted similarly to an empirical cumulative density function). The onion decomposition has a time complexity of $\mathcal{O}(|\mathcal{E}| \log |\mathcal{V}|)$ as reported in [32], which relates to the eigenvector centrality, which is a generic case of the PageRank centrality.

It is important to note that there are several other node, edge, and graph statistics. Most of them were considered to be included in our study but discarded due to their complexity, such as the clustering coefficient and the betweenness or the closeness centrality. We refer the reader of the works of [33,34] for a more robust review of node centralities and other graph-level measures.

2.1.2. Graph kernels

Graph kernels (GK) are one the most used and successful approaches for graph classification. They stand for methods that use (graph) kernel functions, which are similarity functions defined over a pair of graphs, usually passed to a downstream classifier such as SVM [35]. As defined in [36], let $\mathbb{G}$ be a finite set of graphs, $\mathcal{H}_k$ a Hilbert space and $\phi : \mathbb{G} \rightarrow \mathcal{H}_k$. A kernel $\mathcal{K}$ between any two graphs $g, h \in \mathbb{G}$ is denoted as $\mathcal{K}(g, h) = \phi(g) \cdot \phi(h)$, the dot product of their representation in $\mathcal{H}_k$. Such space and feature map ϕ exist only and if only $\mathcal{K}$ is a positive-semidefinite function.

The BON strategy presented in Sub-Section 2.1.1 can be seen as the explicit feature map of an R -Convolution kernel. The R -convolution is a type of kernel based on the comparison of all pairs of substructures of a graph. Any graph vector decomposition can lead to a new graph kernel [37], and many kernels were developed by new ways of decomposing graphs. For instance, another simple yet effective kernel is the histogram vertex kernel [38], where a simple histogram (graph decomposition) is built for every graph counting their node labels.

The use of graph kernels must always be adopted with care due to the time complexity involved in their calculations, even if the explicit calculation of feature maps can be avoided. Besides that, the family of WL kernels [37], based on the Weisfeiler-Lehman Isomorphism Test, tends to be a good baseline for graph classification problems [36]. Nonetheless, there are various graph kernels defined in the literature, and the reader should refer to the works of [36,39] for a broader overview of graph kernel methods.

2.1.3. Other representation methods

There is a vast literature on graph representation for downstream tasks, and several tasks are closely related to each other. Here, other methods are highlighted, such as graph embeddings and graph neural networks.

Graph embeddings and manifold learning. As depicted in [40], manifold learning resides in a class of learning where the main assumption is that high-dimensional data can be sufficiently expressed in a low-dimensional vector. Hence, methods are developed to reduce matrix dimensionality, creating a graph embedding as result. Manifold learning methods are not intrinsically graph problems, but induced into one by the creation of proximity matrices of data points [41]. According

to [42,43], several proximity matrices were proposed, yet all methods share the principle of matrix factorization with some form of graph structure preservation. This produced low-dimensional matrix can be seen as a graph representation (embedding), with each row representing a node. The reader interested in these methods should initially refer to [42,43].

Graph neural networks and other deep learning methods. Graph Neural Networks are currently one of the most popular strategies with respect to graph representation. Since a key feature of deep learning is learning a representation as part of the downstream task learning process, the so-called GNNs define the graph representation as part of the end-to-end learning process, almost as a by-product.

The most common framework is based on the message-passing framework, where the node representation is a learned non-linear function of its neighbors' representations [1,2,5]. As of graph embeddings, most of these groundwork research was done on generating node embeddings that still need some sort of pooling/aggregation/readout operator to create graph-level representation as depicted in [44]. The work of [13] presents a more detailed review of graph-level learning approaches, including those that rely on GNNs.

Nevertheless, there are other ubiquitous graph embeddings that use deep neural network models and have also demonstrated interesting results, but are not conceptually graph neural networks. Such examples are methods based on context generation through random walks like [45–47].

2.2. Weightless classification

The WiSARD [21] classifier is a pattern recognition system [48] derived from the n -tuple classifier of Bledsoe and Browning [49], a weightless neural network. A weightless neural network contrasts with neural networks based on the McCulloch and Pitts neuron model [50] that are the basis of deep neural network systems. Instead, WiSARD was designed to use RAM nodes to memorize co-occurring bit patterns. The fundamental learning process is founded on the presentation of a bit vector whose label is known, being a supervised learning method. Binarization should be performed if the data is not originally in a bit vector format.

During the training phase, each example of the training set is mapped into n -tuples. Usually, this mapping is random and the same for all examples presented to the classifier. Each of these tuples works as an n -bit address of a RAM node of size 2^n , which is marked with a 1, constituting a pattern to be memorized by the network. Groups of RAM nodes, called discriminators, can be organized to match the given label associated with the presented bit vector. Therefore, a discriminator (or a set of them) will store all the information related to a specific class in the classification task. WiSARD models are natural multi-class learners, since the addition of a new output class is simply done by training a discriminator for it.

In order to predict (or classify) an example whose class is to be known, the bit vector is mapped once again into n tuples and presented to the trained classifier. A RAM will be activated if its n -bit address presented as a tuple is written (has appeared on the training set). Each discriminator may have several RAMs activated, and this number represents the score of a given discriminator (or class) for the example to be classified. The output class is the class of the discriminator that has the highest score. Undoubtedly, ties can happen and are randomly broken, which may cause poor classification accuracies.

Further work has expanded the original WiSARD model in order to surpass its limitations, enabling it to deal with more complex problems. The tying problem has led to the Bleaching [51] procedure and its variations, such as the recent model binarization [27] and DAB [52]. Originally designed as a supervised classifier, there are extensions to unsupervised learning [53,54] and regression [55,56]. Its initial architecture has also been studied taking the RAM model

as just an abstraction. For example, Bloom Filters [57] and other lookup table schemes [58] have been used not only without compromising its performance, but, sometimes even enhancing it. Later works have incorporated sophisticated ensemble methods [59,60] and multishot learning via backpropagation [60,61]. Nonetheless, the natural suitability of WiSARD-based models for portability to energy-efficient hardware, such as edge-computing technology or FPGAs, is still verified in all these recent and more sophisticated variants [22,27,60,61].

2.3. Binarization strategies

Binarization/quantization plays a crucial role in WNNs, as it defines the network input. Therefore, how continuous and categorical attributes should be encoded in a bit vector is the subject of many recent works [27,61–64] that used WiSARD-based systems. One of the most adopted encoding algorithms by the WNN community is the thermometer encoding [65], whose name is due to its similarity with a mercury thermometer. A mercury thermometer has a maximum value, a minimum value, and some intermediate marks. The thermometer encoding has B bits to represent a continuous value in a discrete way. Hence, $B - 2$ intermediate values are defined, establishing sequential thresholds or marks. Once a value reaches a certain i th threshold, that i th bit is marked with a 1, resembling the mercury expanding through the thermometer.

The classical thermometer approach is analogous to an equal-width discretization approach [66]. Lately, some works have proposed alternative methods to define the range and intermediate values of a thermometer encoding, such as using standardization (Gaussian) [67] or quantiles, as the Distributive Thermometer [64]. These works have shown that greater expressiveness can be achieved and, therefore, better results. They have also demonstrated that it might be beneficial if the Hamming distance between the binarization result of two inputs is correlated to a chosen distance measure in the original continuous space. That is one of the reasons that the thermometer encoding is widely used.

2.4. KernelCanvas

Depending on the problem of interest, several other encoding schemes have been especially proposed for use in conjunction with WiSARD models. We highlight here one of these methods, called KernelCanvas [28], originally proposed to tackle the time-series problem. The KernelCanvas is a dual binarization and normalization method that tries to mimic the act of painting on a canvas, as exemplified in Fig. 1.

Instead of using a matrix to represent a discrete canvas, KernelCanvas uses $\mathcal{K}$ - a set of kernels — to divide the space similarly to a Voronoi diagram. Each kernel is defined as a randomly sampled point $k \in [-1, 1]^d$, where d equals to the number of dimensions of the canvas and the original input. Any arbitrary size sequence of values $\in \mathbb{R}^d$ can be used as input and each point of the sequence is standardized using the Z-Score (with respect to the sequence) and the $\tanh$ function to fit normalized values in the range of $[-1, 1]^d$. Each point is then mapped to $\kappa = \lambda * |\mathcal{K}|$ kernels that are closer to it on the canvas, where $\lambda \in (0, 1)$ is called the activation rate. The final vector representation is obtained by fixing the number of bits that represents each kernel. Typically, all kernels are represented by the same number of bits β , resulting in a final binary vector of size $B = |\mathcal{K}| * \beta$ bits.

2.5. Related works

Concerning shallow machine learning strategies to graph classification, some approaches must be highlighted. NETSimile [68,69] is a graph representation extraction framework that focuses on local profiles of nodes that are aggregated and then used for downstream tasks. The authors also present an eigenvalue decomposition as a graph feature vector (EIP), in a similar way they later were used in [70].

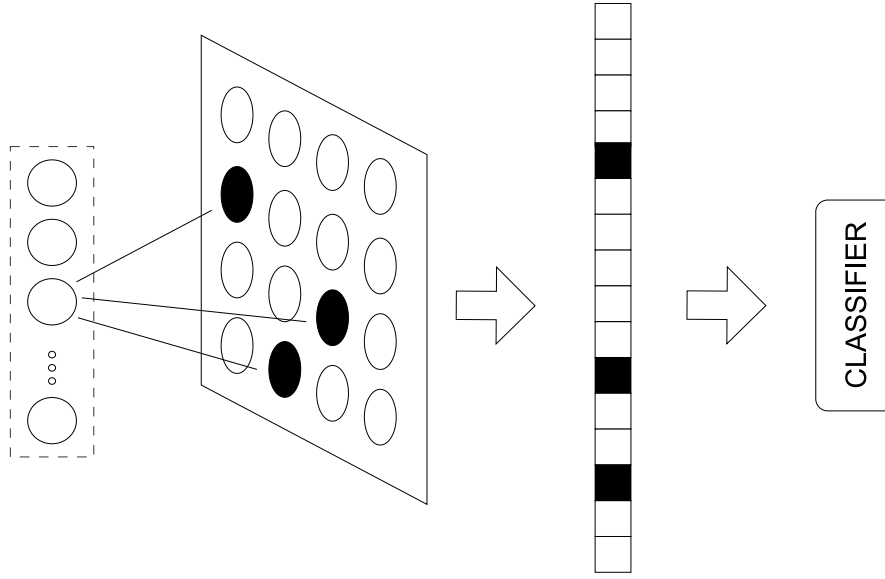


Fig. 1. The big picture of the KernelCanvas procedure. An arbitrary size sequence of values is mapped to its closest kernels in $[-1, 1]^d$. Hence, the canvas analogy: a painter would stroke the canvas several times, with the picture as a composition of all strokes.

The EIP baseline is also related to the works of [71,72] where spectral profiles are derived from a heat process based on the graph Laplacian. Another spectral theory based approach is FGSD [73], where a graph feature vector is built upon the multiset of node spectral pairwise distances.

The notion of neighborhood characteristic functions is studied in [74], where r -scale random walk weighted characteristic functions for node attributed graphs are sampled and aggregated to create whole-graph embeddings. [75] tries to improve on [74], also using aggregations of characteristic functions, but of k -hop sub-graphs using topological similarity jointly based on heat processes. Another related work is [76], where authors create a vector representation using a variety of global graph descriptors. More recently, [77] presents a method using vector representations based on graph kernels, where each input is a stack of different kernel values. In [78], several graph metrics are analyzed to distinguish between real and synthetic networks. Authors in [79] proposed a framework that can be considered a reiteration of [68], reducing the scope to only degree as the metric of interest. Degree-based vector representations were proposed as features for graph comparison also in [80].

Both works of [68,79] are very similar to our work, in the way they use signature vectors as aggregation of node-level descriptors for a graph. Nonetheless, we adopt different classifiers as baselines and our representation is original and solely based on the WiSARD classifier. We also incorporate every attribute of the datasets and the decision to use them or not is a hyperparameter search. Moreover, we include different node descriptors, such as the onion layer number. The only reference that joins both worlds of graph learning and weightless neural networks is [81]. In [81] authors try to classify graphs in the process mining context. The representation and specific context make their attempt not applicable to this work and the datasets here evaluated.

3. A weightless approach to graph classification

Most of the work presented here was done to verify a possible shallow and weightless learning approach to graph classification. Henceforth, we define our task as learning a function that labels a graph G , without a label, with one of the labels in Σ , a predefined set of labels. In other words, this can be described as learning $f \sim l(G)$, where $l : G \rightarrow \Sigma$.

3.1. A shallow weightless architecture

Our learning architecture is derived from the previous work of [82], with three big blocks that can be interpreted as a classic shallow learning approach. As summarized in Fig. 2, the process itself can be thought of as a three-function composition: $f = C \circ Q \circ M$, with a measure gathering step - M , a pre-processing and quantization unit - Q , and a classifier - C , on the edge of it. The measurement gathering step was motivated by the necessity of presenting the graph structure as our learning piece to a final classifier. The whole idea is to build a full graph representation as an aggregation of node and edge attributes.

Although several metrics could have been used, we adopted the previously described graph descriptors: node degree, page-rank centrality, and the onion decomposition. This set of measures has already provided promising results in previous experiments [32,82] without compromising computational complexity.

In order to further expand on [82], we reduce the scope of possible strategies down to two: a KernelCanvas-based attribute aggregation and a histogram-based attribute aggregation. The KernelCanvas approach is used for every continuous attribute, and the histogram is used for categorical values.

The histogram-based approach for categorical values can be defined as a diphasic algorithm. First, for each categorical attribute a in a graph G_k , we map all P_a possible values into P_a bins, and thus, a histogram is created. This histogram is then further binarized using a classic linear thermometer, where the maximum value is the node or edge count (depending on the attribute type), the minimum is 0, and the number of bits per bin is set as a parameter. As an alternative, we also propose a distributive per categorical value histogram. In an analogous manner, for each categorical attribute a in a graph G_k , we map the set $P_a = \{p_{a,1}, \dots, p_{a,|P_a|}\}$ of all possible categorical values of attribute a into a vector $T_{a,k} = [t_{k,1}, \dots, t_{k,|P_a|}]$ of size $|P_a|$, such that $t_{k,i}$ is the percentage of nodes where $a = p_{a,i}$ in G_k . Second, this percentage $t_{k,i}$ is binarized using a distributive thermometer where quantiles are obtained from respective t_i values across all graphs in the training set.

3.2. An updated KernelCanvas procedure

Regarding the KernelCanvas attribute aggregation, every continuous attribute (being edge or node attribute) has its own distribution. In previous works, the empirical complementary cumulative distribution function (ECCDF) was plotted using the KernelCanvas for every

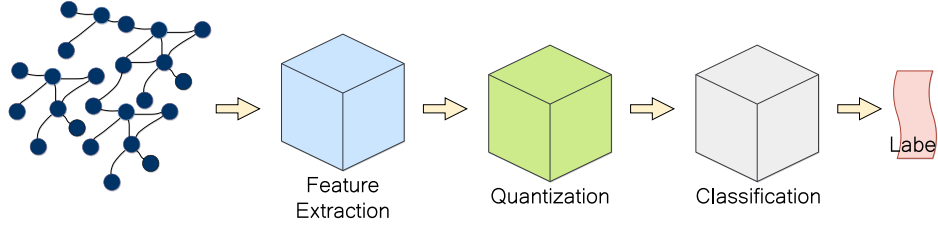


Fig. 2. Our shallow weightless architecture composed of three steps: a feature extraction step - M, a pre-processing and quantization unit - Q, and a classifier - C, on the edge of it.

attribute, except for the onion decomposition, where the onion spectrum was used. Here, we plot joint distributions of the attributes and adapt KernelCanvas using the principles that guide the Distributive Thermometer: linear allocation of bit space.

In order to achieve a better division of the bit space, instead of kernels $\in [-1, 1]^d$, we use kernels $\in [0, 1]^d$. We drop the use of $\tanh$ and the Gaussian normalization. Hence, each dimension gets its normalization (between 0 and 1) applying the empirical cumulative density function (ECDF), as supported by the Probability Integral Transformation [83] (with respect to each dimension). This can be interpreted as marginal distributive thermometers. As an optional additional step, since not every attribute treated as continuous is really continuous (such as node degrees), instead of the commonly used random kernel generation, we use the centroids of a k -means run on the multi-dimensional normalized training data. We call this approach the KernelCanvas++ (KC++) and present its overview in Algorithm 1.

Algorithm 1 KC++ — Transforms a sequence of points $\in \mathbb{R}^d$ into a binary vector $\in \{0, 1\}^B$

Input:
sequence $\triangleright$ Arbitrary size sequence of points $\in \mathbb{R}^d$
trainingData $\triangleright$ A matrix of all training points
 $\mathcal{K}$ $\triangleright$ The kernel set
 κ $\triangleright$ The number of kernels to activate

Output:
input_vector $\triangleright$ The final binarized input vector

```

1: for bit_idx ← 1 to B do
2:   input_vector[bit_idx] ← 0
3: end for
4: for all point ∈ sequence do
5:   for dim ← 1 to d do
6:     newPoint[dim] ← ecdf(point[dim], trainingData[dim])  $\triangleright$  Calculate ECDF using training data w.r.t. dimension
7:   end for
8:   activatedKernels ← findknn(newPoint,  $\mathcal{K}$ ,  $\kappa$ )  $\triangleright$  Find newPoint's  $\kappa$  nearest neighbors ∈  $\mathcal{K}$ 
9:   for all kernel ∈ activatedKernels do
10:    for bit ← 1 to  $\beta$  do
11:      bit_idx ← (kernel - 1) *  $\beta$  + bit
12:      input_vector[bit_idx] ← 1
13:    end for
14:   end for
15: end for

```

3.3. The final representation

Our final representation of a graph — a fixed-size bit vector B — stacks structural components (s) with original non-structural graph attributes (ns), following a simple proportion:

$$B = \alpha B_s \cup (1 - \alpha) B_{ns} \quad (5)$$

where B_s and B_{ns} are binary vectors of all structural and all non-structural attributes stacked, respectively. These vectors are recursively defined as the stacking of quantization Q_i of each attribute i , with

Table 1

Dataset statistics.

dataset	type	# Graphs	#Classes	Avg. Nodes	Avg. Edges
COLLAB [85]	SocNet	5000	3	74.49	2457.78
ENZYMES [19,86]	BioInf	600	6	32.63	62.14
IMDB-M [85]	SocNet	1500	3	13.00	65.94
MUTAG [16,87]	SmlMol	188	2	17.93	19.79
NCII [18,37]	SmlMol	4110	2	29.87	32.30
PROTEINS [19,88,89]	BioInf	1113	2	39.06	72.82

Table 2

Datasets attributes.

dataset	Node labels	Edge labels	Node Attr.	Edge Attr.
COLLAB	—	—	—	—
ENZYMES	Y	—	Y(18)	—
IMDB-M	—	—	—	—
MUTAG	Y	Y	—	—
NCII	Y	—	—	—
PROTEINS	Y	—	Y(1)	—

β_i defining the amount of information of the attribute i in the final representation. The following equation summarizes the definition of B_s :

$$B_s = \bigcup_i Q_{i,s} \quad (6)$$

with B_{ns} being similarly defined.

In our case, in terms of bit amount designations, all z_{ns} non-structural continuous attributes are treated as a single attribute (leading to only one z_{ns} -dimensional KernelCanvas). The same occurs for all structural continuous attributes. Fig. 3 exemplifies the process for two different graphs using only two structural attributes and painting solely the closest kernel (instead of κ kernels). This differs from the framework in [82] where every attribute led to a plot and all the plots were stacked side-by-side to form the final representation. Nonetheless, Eq. (5) empowers us with a model that can be fed with all node and edge attributes originally held by the graphs, processing any node or edge attribute as all node centralities were processed back then.

4. Experimental study

4.1. Datasets and hypotheses

The task of graph classification has brought to life innumerable benchmarks and datasets. In this work we have assessed the model's performance in a subset of TUDatasets [84]. We have selected three groups of graphs: social networks, bioinformatics, and small molecules. These datasets have been used in a variety of recent works regarding graph classification and are considered to be small-size datasets. Tables 1 and 2 summarize every dataset used in this work.

Every non-structural attribute in the original dataset was used in the final input vector. Concerning structural information, the node degree, the page-rank centrality, core-number, and onion-layer were used. The proposed representation was evaluated considering one baseline and

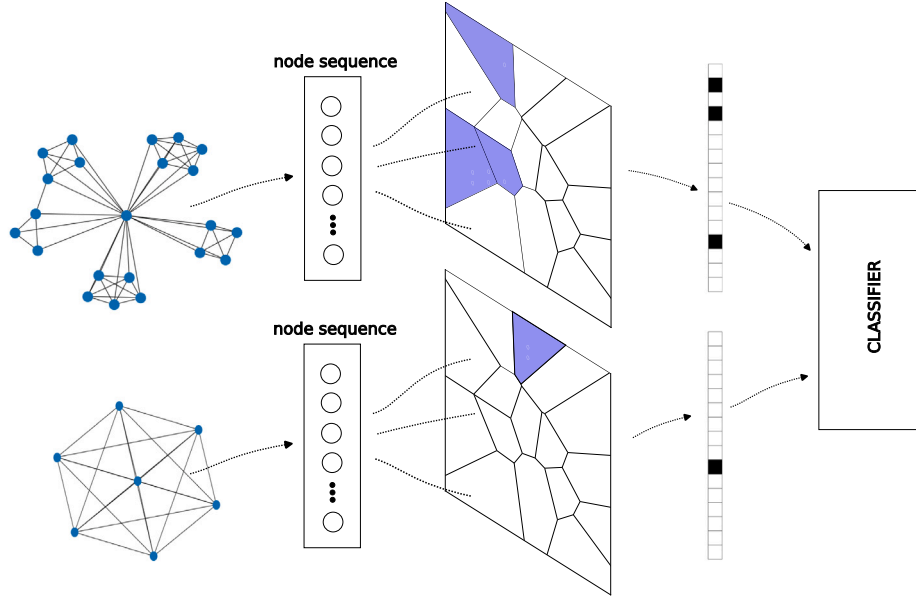


Fig. 3. The KernelCanvas procedure applied to two different graphs using only two structural attributes. Nodes are presented as an arbitrary sequence of points, painting their closest kernel's area. Existing node and edge attributes are encoded in similar fashion and stacked to compose the final input vector.

four main possibilities. A baseline (BASE) used no ECDF normalization with random kernels and the vanilla histogram binarization. A first variation (PROP-1) used the ECDF normalization with the vanilla histogram binarization. Secondly, PROP-2 used ECDF normalization with the alternative histogram binarization proposed in this section. Moreover, PROP-3 used ECDF with K -means kernels. PROP-4 combined both PROP-2 and PROP-3.

Since the proposed model does not specify a final classifier, we have compared the WiSARD (WIS) and some of its evolutions, namely ClusWiSARD (CWIS) and BloomWiSARD (BWIS), with a RandomForest baseline. A committee/ensemble of WiSARD classifiers was also evaluated. Here, we named this strategy as RandomWiSARD (RWIS). We also compared the WNN results to some staple GNN models — GIN [2] and GCN [1], and some other recent GNN models — WLHN [90], a recent model leveraging hyperbolic neural networks and PMLP [91], a graph neural network that is trained as a multilayer perceptron but adopts a message-passing architecture during inference. The node inputs were the same as the WNN, except for edge-level attributes, since the evaluated GNNs do not use them. We adopted a simple mean readout as the final layer of these node-based message passing networks.

4.2. Hyperparameters and implementation

Although we have fixed the M and Q steps, several hyperparameters still had to be stressed. One vital hyperparameter is the size of the bit vector representation B . Another important hyperparameter is α , the rate of structural information versus non-structural information, and β , the inner attribute proportion, which is limited to a minimum amount of information - min_info - another hyperparameter. The B and α parameters serve as proxies to define other several parameters. In our work, we adopted that once α , β_s , and β_{ns} are defined, every β_i will be calculated following a greedy procedure that an attribute i is first uniformly sampled, then its β_i is uniformly sampled from the remaining bits of β_s or β_{ns} , depending on the type of the attribute. For the KernelCanvas, the *activationRate* was kept fixed at 0.07 and the evaluated parameters were $n_kernels$ and $bits_per_kernel$. These were defined in our experiment for each attribute i as a random factorization of β_i . For each histogram, a *thermometer_size* was given. This parameter was fixed as β_i divided by the number of distinct possible values of attribute i .

All classifiers have also their hyperparameters such as *addressSize*, for WiSARD, *addressSize*, *threshold*, *discriminatorsLimit*, and *minScore* for the ClusWiSARD, *addressSize*, *unitHashes*, *unitEntries* for the BloomWiSARD, and *n_estimators*, *criterion*, and *max_features* for the RandomForest, and *addressSize*, *max_samples*, *max_features*, *n_estimators* for RandomWiSARD. Appendix gives a full description of the hyperparameter set used in our experiments.

We performed a ten-fold nested cross-validation, using the same hyperparameter selection methodology as in [92]. A random or grid search was used to search for best validation parameters, fixing in a maximum of $P = 1000$ possibilities within a day of experiments for the WNN and RF models, and $P = 100$ for the GNN models. Also, the search was parallelized as much as possible, using an Intel Xeon E5-2670v3 with 48 cores and 64 GB of RAM. All work was developed building necessary and still unpublished extensions to the *wisardpkg* [24] library. The WLHN model was evaluated using authors' original code¹ and the other GNN models were implemented using the PyTorch Geometric library [93].

5. Results and discussion

5.1. Overall results

We start this section by introducing our main result on the TU-Datasets with no distinction between the variations (BASE, PROP-1, PROP-2, PROP-3, and PROP-4). Table 3 shows our overall best results alongside results for SVM, collected from [39] and the evaluated GNN models. The GNN data for the COLLAB dataset were collected for completion from [90] as our experiments could not finish due to shortage of resources. Both works of [39,90] have an experimental setup comparable to ours, based on [92]. Results show that the proposed shallow architecture leads to very competitive accuracy in comparison to deep models. The WiSARD-based classifiers achieve best overall results in the IMDB-M, ENZYMES, and PROTEINS datasets. Nonetheless, the RandomForest approach tops other two dataset charts, indicating that shallow architectures may beat deep architectures in almost every dataset. In an overall analysis, RandomForest and vanilla WiSARD

¹ <https://github.com/MichailChatzianastasis/WLHN>

Table 3

Performance summary on TUDatasets (mean accuracy over 10-fold evaluations). Best results are highlighted with a black background, second best with a gray background. Accuracy of SVM and GNN models collected from [39] and [90].

	COLLAB	ENZYMES	IMDB-M	MUTAG	NCI1	PROTEINS
SVM	84.5 ± 2.0	67.7 ± 6.5	51.7 ± 4.1	88.3 ± 7.1	86.3 ± 1.6	71.1 ± 4.4
GCN [1]	71.3 ± 2.0	79.6 ± 2.4	65.2 ± 3.0	87.9 ± 6.1	67.6 ± 2.3	74.3 ± 3.5
GIN [2]	75.6 ± 2.3	82.9 ± 1.4	62.6 ± 4.1	84.0 ± 9.4	70.4 ± 2.2	74.2 ± 2.0
WLHN [90]	76.2 ± 2.3	86.1 ± 3.5	65.8 ± 2.6	86.6 ± 10.5	73.2 ± 2.2	74.8 ± 3.0
PMLP [91]	56.4 ± 4.5	83.1 ± 3.6	60.5 ± 2.9	83.5 ± 7.4	56.0 ± 2.9	61.7 ± 4.4
RF	85.7 ± 1.1	90.3 ± 2.9	66.1 ± 4.3	91.0 ± 7.6	74.2 ± 1.5	76.3 ± 4.3
WIS	85.3 ± 0.9	90.2 ± 2.4	66.4 ± 2.9	89.3 ± 7.0	73.5 ± 1.7	76.2 ± 4.8
CWIS	81.7 ± 1.7	89.4 ± 2.9	65.5 ± 3.6	89.8 ± 9.9	73.1 ± 1.7	74.5 ± 3.3
RWIS	80.8 ± 1.9	90.0 ± 2.1	66.2 ± 2.5	88.7 ± 11.0	72.5 ± 1.9	76.9 ± 4.2
BWIS	78.5 ± 1.4	90.5 ± 2.4	66.4 ± 3.2	87.8 ± 6.6	73.0 ± 1.7	76.2 ± 2.8

Table 4

Performance on the COLLAB dataset using different vector input sizes (mean accuracy over 10-fold evaluations). Best results are highlighted with a black background, second best with a gray background. OOT means that results did not complete after 24 h of experiment.

Input size	RF	WIS	CWIS	BWIS	RWIS
2048	84.9 ± 1.1	84.8 ± 1.4	80.5 ± 1.4	75.2 ± 7.3	80.1 ± 1.6
4096	85.7 ± 1.1	84.7 ± 1.2	80.9 ± 2.2	73.9 ± 10.1	80.8 ± 1.9
6144	85.1 ± 1.4	85.1 ± 1.4	80.6 ± 1.9	78.5 ± 1.4	80.6 ± 1.9
8192	85.4 ± 1.0	85.3 ± 0.9	81.1 ± 1.4	OOT	OOT
10240	85.5 ± 0.9	85.3 ± 1.1	81.3 ± 2.3	OOT	OOT
12288	85.6 ± 1.1	85.0 ± 0.9	81.3 ± 1.9	OOT	OOT
14336	85.3 ± 0.8	85.1 ± 1.5	81.2 ± 2.0	OOT	OOT
16384	85.5 ± 1.3	85.3 ± 1.3	81.7 ± 1.7	OOT	OOT

Table 5

Performance on the ENZYMES dataset using different vector input sizes (mean accuracy over 10-fold evaluations). Best results are highlighted with a black background, second best with a gray background.

Input size	RF	WIS	CWIS	BWIS	RWIS
2048	89.6 ± 3.0	88.6 ± 2.1	87.7 ± 2.1	87.5 ± 2.3	88.8 ± 2.8
4096	89.7 ± 2.3	89.2 ± 2.7	88.1 ± 3.1	89.8 ± 3.0	87.1 ± 3.6
6144	90.3 ± 2.9	89.7 ± 2.3	89.1 ± 3.2	89.2 ± 2.7	88.9 ± 1.9
8192	90.2 ± 2.4	89.2 ± 2.6	89.1 ± 2.6	89.1 ± 2.1	88.8 ± 2.3
10240	89.7 ± 1.5	90.1 ± 2.8	88.6 ± 2.8	89.3 ± 3.3	89.8 ± 2.4
12288	89.7 ± 2.7	89.0 ± 1.8	89.4 ± 2.9	89.6 ± 2.5	89.7 ± 1.9
14336	89.2 ± 2.5	89.8 ± 2.2	89.3 ± 2.8	89.4 ± 2.4	88.6 ± 3.2
16384	89.9 ± 2.3	90.2 ± 2.4	88.9 ± 2.7	90.5 ± 2.4	90.0 ± 2.1

present the best results. Some works [94,95] debate if the structure is relevant to some of the datasets being evaluated. Although being irrelevant to some, the representation presented here is powerful in both cases. Other datasets and classifiers could be evaluated and there is still room for hyperparameter optimization, since no optimization scheme was adopted besides the random/grid search.

5.2. Representation size analysis

This section presents an analysis of the performance of the WNN and RF models for different input sizes in every dataset, ranging from 2048 to 16,384 bits. Overall, there is no silver bullet when deciding the number of bits of the input vector. Every dataset appears to be a singular problem, as Tables 4 to 9 show. Apart from the COLLAB dataset, where vanilla WiSARD and RandomForest appear to be clear winners, performance for all classifiers seems to be similar, within a standard deviation margin. Considering the NCI1 dataset, there is tiny evidence that greater representations may work better, since best results are closer to the upper limit of the grid search considering the input size. Increasing input size is also a double-edged sword,

particularly when dealing with larger datasets, such as the COLLAB. Larger representation leads to both higher computing time and resource consumption, which may incur in execution errors of timeout and/or lack of memory.

5.3. Feature importance analysis

One key aspect of our pipeline is the search for the best representation as part of the hyperparameter search. In Fig. 4 we present a summary of the best trade-offs between structural and non-structural features that were found for each dataset in terms of a percentage of the size of the input vector with respect to the WNN and RF models. For the ENZYMES dataset, we can see a great majority of non-structural attributes prevailing. The scenario is similar but not so extreme for the MUTAG and the NCI1 datasets, where there is some predominance of non-structural attributes. It is relevant to notice that in the latter there is only one non-structural attribute (node labels). Yet, they seem to play a key role when differentiating the classes. An opposite behavior happens in the PROTEINS dataset where structural information appears to be more important than non-structural information. IMDB-M and

Table 6

Performance on the IMDB-M dataset using different vector input sizes (mean accuracy over 10-fold evaluations). Best results are highlighted with a black background, second best with a gray background.

Input size	RF	WIS	CWIS	BWIS	RWIS
2048	65.2 ± 3.5	65.0 ± 2.6	62.8 ± 2.6	65.3 ± 2.4	65.2 ± 2.9
4096	64.8 ± 3.4	65.1 ± 2.6	64.2 ± 3.4	65.4 ± 3.2	64.1 ± 3.6
6144	65.8 ± 3.0	65.6 ± 4.1	64.4 ± 3.7	65.3 ± 2.5	65.1 ± 2.8
8192	65.8 ± 3.6	65.9 ± 3.8	64.1 ± 4.0	65.8 ± 2.2	65.3 ± 1.9
10240	65.6 ± 3.6	65.5 ± 3.9	64.6 ± 2.8	65.8 ± 3.2	65.8 ± 3.3
12288	66.0 ± 3.7	66.4 ± 2.9	65.2 ± 4.3	66.4 ± 3.2	65.6 ± 3.5
14336	65.9 ± 3.9	65.4 ± 3.1	65.5 ± 3.6	65.6 ± 2.7	64.2 ± 4.0
16384	66.1 ± 4.3	65.8 ± 3.9	65.0 ± 3.4	65.6 ± 3.0	66.2 ± 2.5

Table 7

Performance on the MUTAG dataset using different vector input sizes (mean accuracy over 10-fold evaluations). Best results are highlighted with a black background, second best with a gray background.

Input size	RF	WIS	CWIS	BWIS	RWIS
2048	86.8 ± 7.2	87.7 ± 7.4	85.5 ± 9.8	85.6 ± 9.0	85.1 ± 8.3
4096	87.3 ± 10.7	86.1 ± 7.0	87.7 ± 8.2	85.0 ± 6.7	85.1 ± 8.5
6144	87.9 ± 11.1	86.1 ± 11.1	86.1 ± 8.3	85.1 ± 6.5	84.1 ± 9.3
8192	89.4 ± 7.4	88.7 ± 8.0	88.2 ± 9.3	85.6 ± 7.9	86.2 ± 7.5
10240	91.0 ± 7.6	87.7 ± 6.8	88.2 ± 8.1	86.2 ± 7.1	82.0 ± 9.9
12288	89.4 ± 8.2	86.6 ± 8.2	87.1 ± 8.4	87.8 ± 6.6	81.9 ± 7.9
14336	87.3 ± 8.7	89.3 ± 7.0	86.6 ± 7.0	87.2 ± 6.8	89.4 ± 5.6
16384	88.3 ± 6.4	88.7 ± 8.5	89.8 ± 9.9	86.1 ± 8.1	87.8 ± 5.0

Table 8

Performance on the NCI1 dataset using different vector input sizes (mean accuracy over 10-fold evaluations). Best results are highlighted with a black background, second best with a gray background.

Input size	RF	WIS	CWIS	BWIS	RWIS
2048	74.7 ± 1.9	70.3 ± 2.1	69.3 ± 2.2	69.2 ± 2.4	70.8 ± 2.1
4096	74.8 ± 1.4	72.0 ± 2.3	70.3 ± 1.7	69.9 ± 2.4	70.2 ± 1.4
6144	74.2 ± 2.7	73.0 ± 2.6	70.2 ± 2.9	71.8 ± 1.4	71.5 ± 2.0
8192	75.2 ± 2.5	72.9 ± 2.6	70.9 ± 2.0	71.9 ± 1.5	71.2 ± 2.6
10240	74.6 ± 3.1	72.5 ± 3.0	72.3 ± 2.3	71.9 ± 1.6	71.4 ± 2.1
12288	75.1 ± 2.4	73.9 ± 2.2	72.5 ± 1.2	72.3 ± 2.2	71.5 ± 2.2
14336	75.8 ± 2.3	73.6 ± 2.1	73.1 ± 1.7	72.1 ± 3.1	70.6 ± 1.3
16384	75.8 ± 2.9	72.1 ± 2.9	72.1 ± 2.0	73.0 ± 1.7	72.5 ± 1.9

Table 9

Performance on the PROTEINS dataset using different vector input sizes (mean accuracy over 10-fold evaluations). Best results are highlighted with a black background, second best with a gray background.

Input size	RF	WIS	CWIS	BWIS	RWIS
2048	74.8 ± 3.5	73.1 ± 3.7	71.0 ± 2.3	73.2 ± 4.5	74.9 ± 4.9
4096	73.5 ± 5.4	73.8 ± 4.7	72.8 ± 3.5	73.6 ± 3.9	75.5 ± 3.5
6144	76.3 ± 4.3	73.9 ± 5.0	72.2 ± 4.4	73.8 ± 3.6	76.2 ± 5.0
8192	73.8 ± 4.9	74.1 ± 4.7	73.2 ± 3.9	74.8 ± 4.1	74.8 ± 4.9
10240	73.2 ± 4.6	75.6 ± 5.8	74.5 ± 3.3	74.0 ± 3.6	76.9 ± 4.2
12288	73.7 ± 4.7	73.7 ± 5.3	72.6 ± 4.1	74.9 ± 3.9	75.9 ± 3.6
14336	70.8 ± 5.3	71.4 ± 5.5	72.5 ± 4.2	73.5 ± 4.6	75.9 ± 4.6
16384	72.9 ± 5.9	76.2 ± 4.8	73.0 ± 3.7	76.2 ± 2.8	76.3 ± 5.1

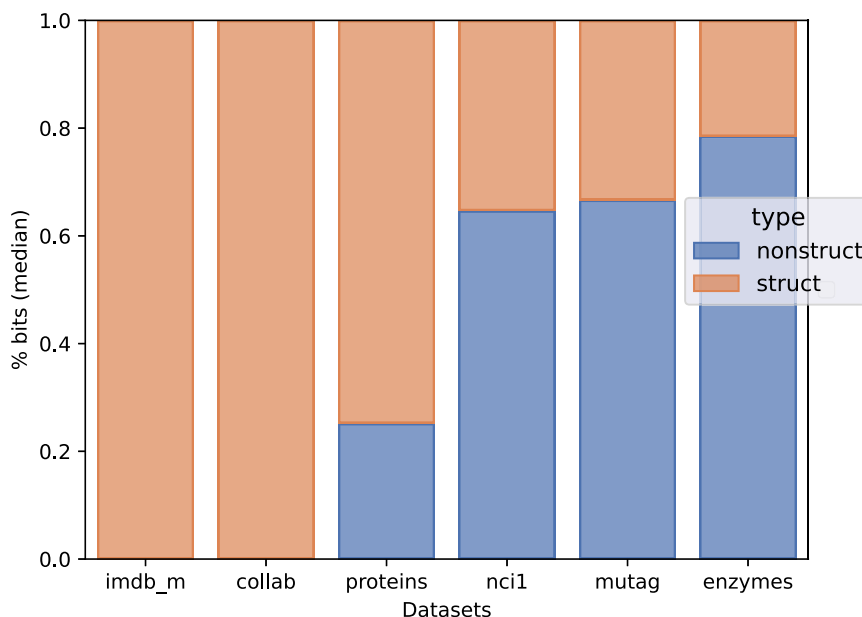


Fig. 4. Trade-offs between structural and non-structural attributes in the best result for all datasets (median of all folds). Datasets are ordered by structural attributes' usage in final classifier.

COLLAB datasets leverage only structural attributes. As a concluding remark, there is no clear preset of attributes that works better overall. Every dataset is a unique problem of its own. The most important is to ensure that a final model can absorb these differences.

5.4. Resource usage analysis

In this section, we evaluate resource consumption using different metrics. Data in Tables 10–12 were collected using PBS² trace reports for the whole process, including hyperparameter search, training, model selection, and testing, except for the final model size where the size of the serialized classifier is reported. We present results for WNN, RandomForest and the selected GNN baselines. This evaluation was carried out fixing the number of hyperparameter combinations(P) to 100 within a day to establish a fair comparison. In Table 10, results show that vanilla WiSARD is generally less energy demanding than the baseline RandomForest classifier, except for cases where it follows RF by a small margin. That fact does not hold though for its variations, where BloomWisard can be up to twenty times more energy demanding (in IMDB-M dataset), being as energy demanding as some GNN models. Moreover, the GNN models were the worst concerning their energy demands.

Table 11 presents average final model sizes considering only the best classifiers. Taking into consideration the recent advances of BloomWiSARD model, this is where it shines, providing the most compact WNN model. On the hand, the RWIS approach is the greatest, since it can be seen as an ensemble of models of relative big size. The evaluated GNN were not as deep as other deep network models, resulting in very compact final models, with few kilobytes in size, topping the charts. Concluding, Table 12 reveals the memory consumption of all models, with vanilla WiSARD being generally the less demanding classifier. On the other hand, GNN models are the most demanding, imposing roughly twice the memory requirements of a WNN model. It is important to note that these memory consumption values are partially due to high data parallelism leveraged for hyperparameter search, not being strictly this high if a single-threaded model was trained.

Following our resource usage analysis, we break down the time spent on every key phase of the study. Table 13 shows time spent on hyperparameter search, training the full model and the inference phase for all evaluated datasets. It is important to point out that our current implementations are not production-ready, and time comparisons can be misleading. The results in Table 13 indicate that the vanilla WiSARD models spent less time searching for hyperparameters and training. Overall, the GNN are faster for inference, followed by the RandomForest baseline. These results are partially explained by a bottleneck caused by the preprocessing of the inputs through the pipeline, and the bleaching technique, which can still be improved with approaches like the one in [52].

5.5. KernelCanvas update analysis

Another aspect of our work are the proposed additions to the KernelCanvas fundamentals. Figures from 5 to 10 present the results in terms of accuracy by the strategy implemented in KernelCanvas and the binarization steps, fixing the WiSARD base classifier as the classifier. Starting with the COLLAB dataset, Fig. 5 shows only three alternatives (BASE, PROP-1, PROP-4). This happens because there is no histogram-based attribute in this dataset (the same applies for Fig. 7). As for the results, the chart shows that PROP-1 is clearly a best alternative in terms of accuracy, followed by PROP-4 and then the baseline. The same difference to the baseline is seen in Fig. 7, where PROP-1 and PROP-4 have similar performance. Regarding the ENZYMES dataset, Fig. 6 exhibits all five KernelCanvas' hypotheses, with PROP-1 once again topping the charts. PROP-3 seems to be the worst in performance, with others performing indistinguishably in the middle. Fig. 8 brings accuracies for the MUTAG dataset, where all hypotheses seem to achieve almost the same results. Results for the NCI1 dataset are presented in Fig. 9. This figure illustrates PROP-1 and BASE performing similarly at the top, with the other three approaches being in a lower cut. Finally, 10 presents results for the PROTEINS dataset, where the only distinct result is the worse accuracy assessed by BASE. The global image brought by the results in this section is that PROP-1 is the to-go strategy, as the others proposals did not perform as expected.

² <https://altair.com/pbs-professional/>

Table 10

Average energy consumption (in Wh) on TUDatasets. Best results are highlighted with a black background, second best with a gray background.

	COLLAB	ENZYMES	IMDB-M	MUTAG	NCI1	PROTEINS
RF	1342.3 ± 857.3	41.9 ± 15.1	45.3 ± 24.3	4.3 ± 1.7	224.1 ± 101.5	52.8 ± 14.5
WIS	1311.6 ± 823.3	42.0 ± 15.9	17.7 ± 7.5	3.3 ± 1.7	215.6 ± 103.0	53.1 ± 15.6
CWIS	1365.2 ± 858.6	46.1 ± 17.0	38.4 ± 19.4	4.2 ± 2.1	231.5 ± 107.3	59.0 ± 17.8
RWIS	1483.9 ± 996.8	107.5 ± 41.7	128.5 ± 70.2	14.0 ± 6.5	520.2 ± 236.3	130.5 ± 58.1
BWIS	1688.7 ± 1030.6	227.3 ± 97.5	207.1 ± 115.4	18.4 ± 9.7	574.9 ± 263.5	181.3 ± 75.7
GCN	OR	168.8 ± 3.7	323.7 ± 9.8	36.4 ± 2.2	831.1 ± 44.7	344.5 ± 9.4
GIN	OR	142.3 ± 4.3	257.3 ± 15.2	35.2 ± 2.2	800.0 ± 46.8	277.4 ± 2.4
WLHN	OR	671.4 ± 17.7	749.6 ± 39.6	127.2 ± 5.8	3901.9 ± 217.2	1377.9 ± 26.6
PMLP	3005.6 ± 800.2	93.7 ± 5.8	179.9 ± 12.6	27.1 ± 0.9	542.1 ± 19.8	185.6 ± 7.0

Table 11

Average final model size (in MB) on TUDatasets. Best results are highlighted with a black background, second best with a gray background. BWIS results for the COLLAB must be taken with care since only smaller models completed the experiment.

	COLLAB	ENZYMES	IMDB-M	MUTAG	NCI1	PROTEINS
RF	20.2 ± 3.6	6.0 ± 1.1	2.8 ± 1.0	0.6 ± 0.1	21.6 ± 4.8	5.0 ± 1.2
WIS	19.9 ± 11.5	5.6 ± 5.3	1.6 ± 1.0	0.8 ± 1.8	3.7 ± 1.7	3.3 ± 2.3
CWIS	19.5 ± 11.6	5.4 ± 2.9	2.8 ± 1.6	1.1 ± 0.6	4.7 ± 2.5	4.2 ± 2.8
RWIS	220.9 ± 119.8	113.4 ± 70.0	39.7 ± 34.0	8.4 ± 6.6	78.3 ± 44.0	51.7 ± 42.2
BWIS	2.5 ± 11.1	0.7 ± 0.4	0.3 ± 0.2	0.2 ± 0.1	0.1 ± 0.1	0.3 ± 0.1
GCN	OR	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
GIN	OR	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
WLHN	OR	0.2 ± 0.0	0.1 ± 0.0	0.1 ± 0.0	0.2 ± 0.0	0.1 ± 0.0
PMLP	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0

Table 12

Average virtual memory usage on TUDatasets (in GB). Best results are highlighted with a black background, second best with a gray background.

	COLLAB	ENZYMES	IMDB-M	MUTAG	NCI1	PROTEINS
RF	39.1 ± 3.9	11.8 ± 0.2	11.8 ± 0.6	10.7 ± 0.1	14.1 ± 0.6	12.0 ± 0.3
WIS	39.7 ± 4.1	11.5 ± 0.3	11.2 ± 0.4	10.6 ± 0.1	13.2 ± 0.8	11.6 ± 0.4
CWIS	40.4 ± 3.6	11.8 ± 0.4	11.4 ± 0.5	10.8 ± 0.2	13.2 ± 0.6	11.8 ± 0.4
RWIS	49.5 ± 6.3	31.5 ± 13.0	20.7 ± 5.5	14.7 ± 2.0	20.0 ± 3.9	20.9 ± 6.1
BWIS	41.7 ± 1.8	17.7 ± 5.4	13.4 ± 2.0	12.2 ± 0.9	13.9 ± 1.2	12.9 ± 1.0
GCN	OR	24.4 ± 0.0	24.4 ± 0.0	23.9 ± 0.1	25.7 ± 0.1	24.7 ± 0.0
GIN	OR	24.4 ± 0.0	24.4 ± 0.1	23.9 ± 0.1	25.7 ± 0.1	24.6 ± 0.0
WLHN	OR	24.4 ± 0.0	24.2 ± 0.0	23.6 ± 0.0	25.7 ± 0.0	24.8 ± 0.0
PMLP	74.6 ± 0.0	24.3 ± 0.0	24.4 ± 0.1	23.9 ± 0.1	25.7 ± 0.1	24.6 ± 0.0

Table 13

Average time (in seconds per fold) spent on hyperparameter search, training and inference on TUDatasets. Best results are highlighted with a black background, second best with a gray background.

	COLLAB			ENZYMES			IMDB-M		
	Search	Training	Inference	Search	Training	Inference	Search	Training	Inference
RF	2368.1 ± 1354.4	285.2 ± 202.8	20.8 ± 11.9	66.6 ± 25.2	9.8 ± 3.7	0.8 ± 0.3	73.5 ± 37.4	8.2 ± 5.2	0.2 ± 0.1
WIS	2296.3 ± 1312.9	279.4 ± 205.1	21.5 ± 12.5	66.6 ± 26.5	9.7 ± 3.9	0.9 ± 0.3	27.5 ± 11.6	3.3 ± 1.4	0.5 ± 0.3
CWIS	2413.0 ± 1364.2	276.2 ± 207.3	21.3 ± 12.7	73.6 ± 28.2	10.1 ± 4.1	1.0 ± 0.3	58.4 ± 27.3	4.7 ± 2.2	2.6 ± 1.7
RWIS	2704.7 ± 1320.8	331.1 ± 204.3	38.6 ± 15.9	150.7 ± 56.4	19.1 ± 8.8	7.7 ± 5.8	175.7 ± 89.8	13.0 ± 7.6	8.5 ± 5.7
BWIS	3324.2 ± 1569.8	394.2 ± 197.1	20.4 ± 12.4	360.7 ± 154.0	62.1 ± 37.4	1.2 ± 0.4	317.8 ± 171.4	50.9 ± 31.5	0.6 ± 0.3
GCN	OR	OR	OR	202.2 ± 2.1	18.8 ± 1.1	0.2 ± 0.0	407.9 ± 5.3	37.9 ± 4.6	0.6 ± 0.1
GIN	OR	OR	OR	173.8 ± 2.4	17.8 ± 2.2	0.2 ± 0.0	346.9 ± 7.5	36.4 ± 3.3	0.5 ± 0.1
WLHN	OR	OR	OR	834.1 ± 7.1	95.2 ± 4.7	0.3 ± 0.0	1003.9 ± 27.7	100.5 ± 11.0	0.6 ± 0.0
PMLP	5286.1 ± 35.1	596.2 ± 19.0	54.2 ± 0.6	125.2 ± 0.9	12.0 ± 0.4	0.2 ± 0.0	247.8 ± 5.8	23.8 ± 2.8	0.5 ± 0.0

	MUTAG			NCI1			PROTEINS		
	Search	Training	Inference	Search	Training	Inference	Search	Training	Inference
RF	6.6 ± 2.6	0.8 ± 0.4	0.1 ± 0.0	382.0 ± 176.5	51.3 ± 25.9	3.6 ± 1.4	86.6 ± 25.2	11.6 ± 3.6	0.9 ± 0.3
WIS	4.9 ± 2.7	0.5 ± 0.4	0.1 ± 0.0	372.0 ± 175.1	48.7 ± 25.5	3.8 ± 1.4	86.3 ± 25.5	11.2 ± 3.6	1.0 ± 0.3
CWIS	6.2 ± 3.2	0.7 ± 0.4	0.1 ± 0.1	403.6 ± 182.9	49.8 ± 25.7	4.0 ± 1.4	96.0 ± 28.3	11.9 ± 4.0	1.2 ± 0.4
RWIS	22.1 ± 9.5	1.4 ± 1.0	0.4 ± 0.3	769.1 ± 341.2	104.2 ± 53.5	21.1 ± 12.7	189.5 ± 75.0	21.9 ± 9.8	6.3 ± 4.3
BWIS	29.0 ± 15.5	3.9 ± 2.8	0.1 ± 0.1	1034.8 ± 466.2	109.6 ± 58.0	4.1 ± 1.5	288.9 ± 117.9	47.5 ± 23.1	1.2 ± 0.3
GCN	47.0 ± 0.7	4.6 ± 0.5	0.0 ± 0.0	1075.0 ± 20.4	110.3 ± 12.1	1.0 ± 0.0	420.9 ± 12.1	31.1 ± 3.7	0.5 ± 0.0
GIN	44.8 ± 0.9	4.3 ± 0.7	0.1 ± 0.0	988.2 ± 24.2	104.4 ± 15.1	1.0 ± 0.1	336.9 ± 4.5	33.0 ± 1.6	0.5 ± 0.0
WLHN	168.6 ± 2.2	13.9 ± 2.7	0.0 ± 0.0	5201.5 ± 60.2	609.7 ± 33.2	1.3 ± 0.1	1788.7 ± 16.6	176.6 ± 8.0	0.6 ± 0.0
PMLP	35.6 ± 0.5	3.1 ± 0.3	0.0 ± 0.0	739.8 ± 10.8	68.3 ± 5.4	1.0 ± 0.1	235.0 ± 5.7	18.7 ± 2.5	0.5 ± 0.0

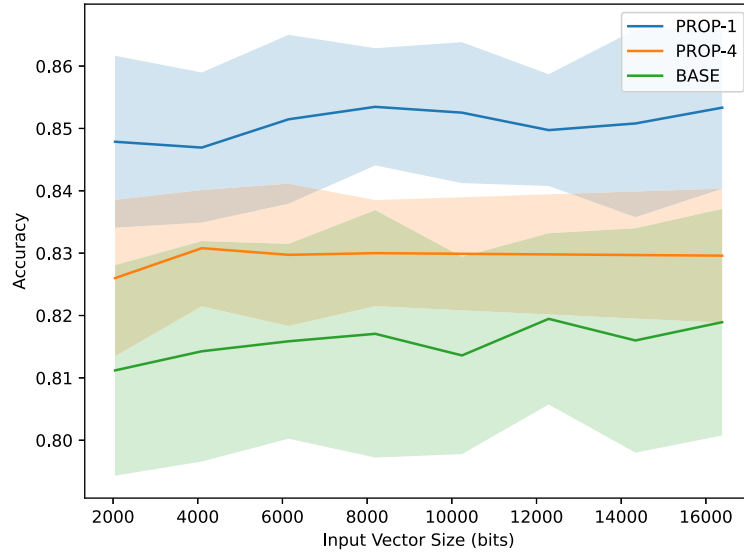


Fig. 5. Accuracy of different KernelCanvas strategies on the COLLAB dataset.

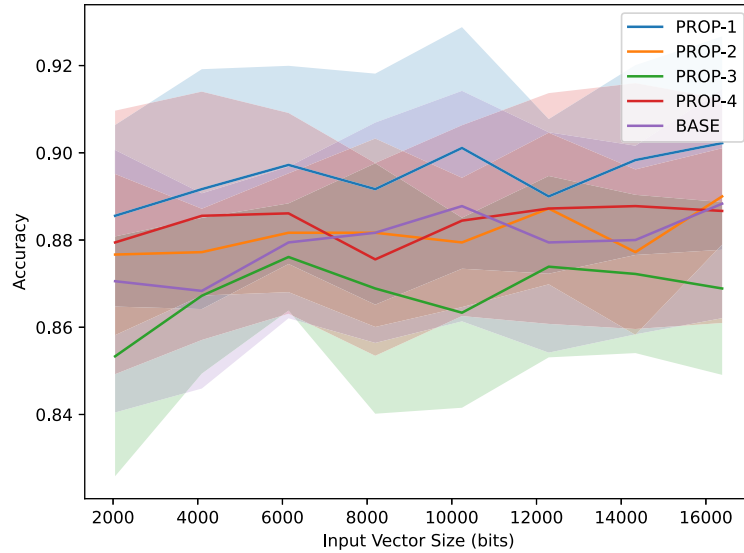


Fig. 6. Accuracy of different KernelCanvas strategies on the ENZYMES dataset.

6. Conclusion

This work has proposed and explored a weightless learning architecture for graph classification. Graph classification is a topic that has recently drawn the attention of the machine-learning community. It is still an open problem, especially regarding lighter and greener solutions, since graph neural networks (and deep learning architectures) dominate the current scenario, encouraging experimentation with lighter classifiers such as the WiSARD-based ones. In order to accompany these flyweight classifiers, we enhanced traditional shallow learning techniques for graphs, such as the bag-of-nodes, a simple form of graph kernels. These reinforce the greener and lighter aspects of our solution.

Our model leveraged local and global features, aggregating and packing them into binary vectors akin to WiSARD-based classifiers. This was possible through the development of new dedicated quantization strategies. These included a proposed update for the KernelCanvas, a widely adopted quantization method for WNNs. We evaluated the

performance of our model in established real network datasets. Although results cannot beat all state-of-the-art methods in terms of accuracy, our primary goal was fulfilled: confirm that more straightforward and cheaper architectures can lead to comparable and competitive performance.

The representation provided here can also benefit other classifiers, such as the tree-based RandomForest classifier, and researchers can explore various other classifier options. The quantization field has seen recent advancements, and numerous contemporary approaches may apply to this problem. There is still ample room to explore pooling operations, which play significant roles in deep learning and graph-level problems. Many attributes remain untapped and could be leveraged, albeit with some complexity restrictions. Additionally, specific attributes were not utilized but could be considered. The literature offers other numerous datasets suitable for evaluating our framework, particularly regarding its generalization and scalability, paving the way for future research endeavors.

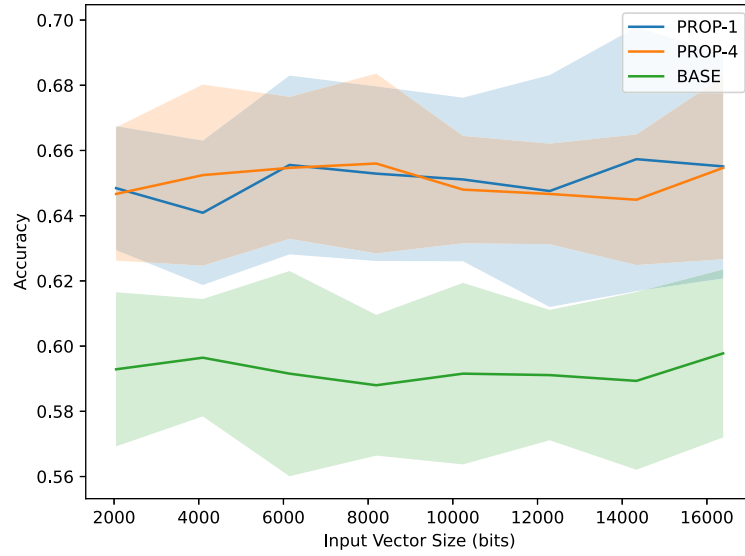


Fig. 7. Accuracy of different KernelCanvas strategies on the IMDB-M dataset.

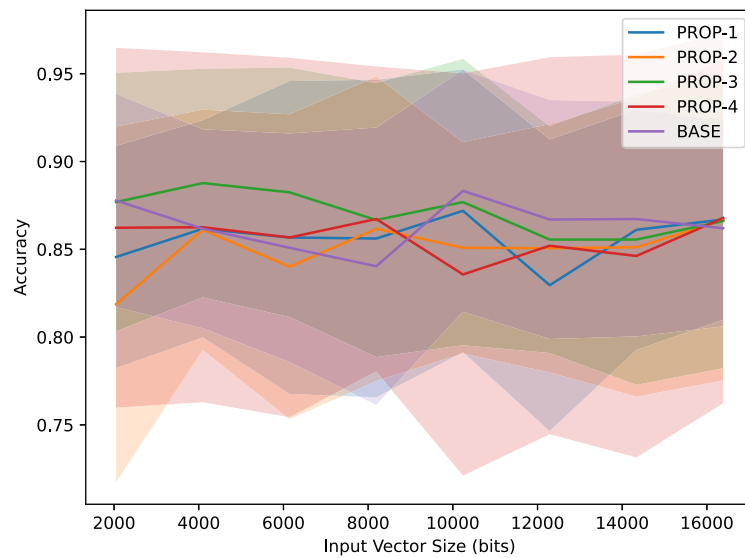


Fig. 8. Accuracy of different KernelCanvas strategies on the MUTAG dataset.

CRedit authorship contribution statement

Raul Bezerra Barbosa: Writing – original draft, Software, Methodology, Conceptualization. **Diego Leonel Cadette Dutra:** Writing – review & editing, Supervision, Methodology. **Priscila Machado Vieira Lima:** Writing – review & editing, Supervision, Methodology. **Diego Carvalho:** Writing – review & editing, Supervision, Methodology. **Felipe Maia Galvão França:** Writing – review & editing, Supervision, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially supported by NACAD/COPPE (UFRJ), DIPPG/CEFET-RJ, Capes, Brazil, CNPq, Brazil, FAPERJ, Brazil, and Project Route25: 02/C05-i01.01/2022.PC645463824-00000063.

Appendix. Hyperparameter settings

Table A.14 presents all hyperparameters searched in our work for the evaluated classifiers and other (hyper) parameters used in our work. ClusWiSARD parameters are the union of all WiSARD parameters plus the ones presented here. Parameters for RandomForest are the default parameters in the scikit-learn python library (version 1.1.1), except where noted. Parameters for GNN models are the default in the Pytorch Geometric Library (version 2.6.1), except where noted.

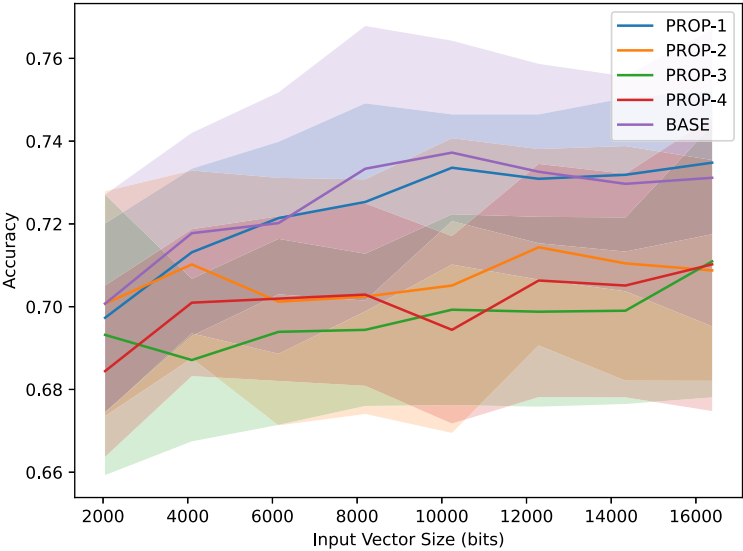


Fig. 9. Accuracy of different KernelCanvas strategies on the NCI1 dataset.

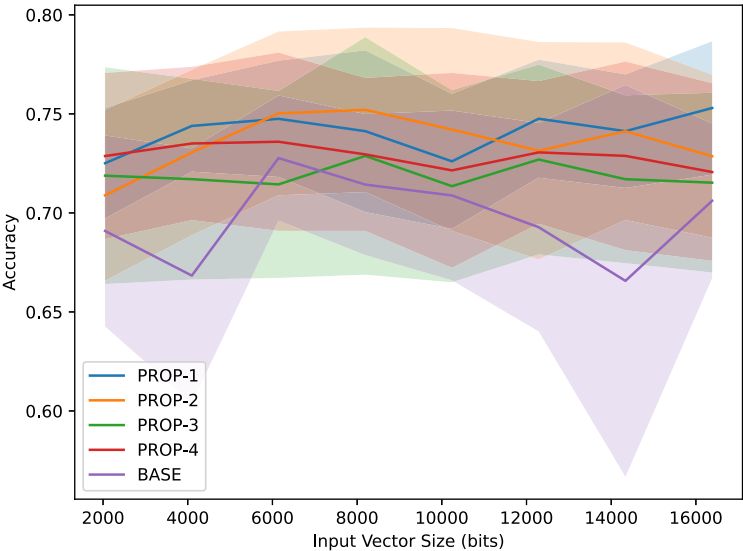


Fig. 10. Accuracy of different KernelCanvas strategies on the PROTEINS dataset.

Table A.14

Hyperparameters used in our work for all classifiers and search procedures.

	Parameter	Description	Search Space ^a
WiSARD parameters	<i>addressSize</i>	Number of bits used for an address in RAM (tupleSize)	{2, 16, 32, 48, 64}
	<i>classificationMethod</i>	How ties are broken	Bleaching
	<i>mappingGenerator</i>	How the inputs are broken into tuples	RandomMapping
	<i>completeAddressing</i>	If address should be completed in case there is a remainder in the division $\text{inputVectorSize}/\text{tupleSize}$	true
	<i>balanced</i>	If the number of training examples for a class should be considered when classifying	false
	<i>ignoreZero</i>	If zero-only address should be discarded when classifying	false
ClusWiSARD parameters	<i>monoMapping</i>	If the mapping is shared between all classes	true
	<i>base</i>	Base of input vector	2 (binary vector)
	<i>minScore</i>	Minimum score to accept a training example	{0.5, 0.7, 0.9}
RandomWiSARD parameters	<i>threshold</i>	Damping factor for training example acceptance	{0.1, 0.2, 0.5, 1}
	<i>discriminatorsLimit</i>	Maximum number of discriminators that can be created for a single class	{1, 5, 10}
	<i>addressSize</i>	Number of bits used for an address in RAM (tupleSize)	{2, 4, 8, 16, 32}
	<i>n_estimators</i>	Number of estimators used in the bagging	{10, 100, 200}
BloomWiSARD parameters	<i>max_samples</i>	Fraction of samples used to train each base estimator	{0.1, 0.5, 1}
	<i>max_features</i>	Fraction of features used in each base estimator	{“log2”, “sqrt”, 0.1, 0.5, 1}
	<i>unitHashes</i>	Number of hash functions	{1, 2, 3, 4, 5, 6}
RandomForest parameters	<i>unitEntries</i>	Number of entries per bloom filter	{128, 256, 512, 1024, 2048, 4096, 8192}
	<i>addressSize</i>	Number of bits used for an address in RAM (tupleSize)	{2, 4, 8, 16, 32, 48, 64}
	<i>n_estimators</i>	Number of trees	{10, 100, 200}
GNN parameters	<i>criterion</i>	Function to evaluate the quality of the split {“gini”, “entropy”}	{“gini”, “entropy”}
	<i>max_features</i>	Maximum features used when searching for the best split	{“sqrt”, “log2”}
	<i>layers</i>	Number of hidden layers	{2, 3, 4}
Other parameters	<i>batch_size</i>	Number of graphs per iteration	64
	<i>optimizer</i>	Optimizer used during training	Adam
	<i>learning_rate</i>	Learning rate used for optimization	{1e-1, 1e-3, 1-6}
	<i>hidden_units</i>	Hidden units per hidden layer	{32, 64}
	<i>num_epochs</i>	Number of epochs	{100, 200}
	<i>dropout</i>	Dropout rate	{0, 0.25, 0.5}
Other parameters	<i>varianceThreshold</i>	A variance threshold	{None, 0}
	<i>P</i>	Maximum of combinations evaluated in a day	1000
	<i>min_info</i>	Minimum β_i - amount of bits for an attribute	{64}
	<i>α</i>	Rate of structural information	{0.1, 0.25, 0.4, 0.5, 0.6, 0.75, 0.9}

^a A set denotes a grid of evaluated hyperparameter values and a constant value represents a fixed hyperparameter value.

Data availability

Data will be made available on request.

References

- [1] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, 2016, [arXiv:1609.02907](https://arxiv.org/abs/1609.02907) [cs.LG].
- [2] K. Xu, W. Hu, J. Leskovec, S. Jegelka, How powerful are graph neural networks? 2019, [http://dx.doi.org/10.48550/arXiv.1810.00826](https://arxiv.org/abs/10.48550/arXiv.1810.00826), [arXiv:1810.00826](https://arxiv.org/abs/1810.00826) [cs, stat].
- [3] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, 2018, [http://dx.doi.org/10.48550/arXiv.1710.10903](https://arxiv.org/abs/10.48550/arXiv.1710.10903), [arXiv:1710.10903](https://arxiv.org/abs/1710.10903) [cs, stat].
- [4] J. Gilmer, S.S. Schoenholz, P.F. Riley, O. Vinyals, G.E. Dahl, Neural message passing for quantum chemistry, in: Proceedings of the 34th International Conference on Machine Learning - Vol. 70, ICML '17, JMLR.org, 2017, pp. 1263–1272.
- [5] F. Scarselli, M. Gori, A.C. Tsoi, M. Hagenbuchner, G. Monfardini, The graph neural network model, IEEE Trans. Neural Netw. 20 (1) (2009) 61–80, [http://dx.doi.org/10.1109/TNN.2008.2005605](https://arxiv.org/abs/10.1109/TNN.2008.2005605).
- [6] A. Micheli, Neural network for graphs: A contextual constructive approach, IEEE Trans. Neural Netw. 20 (3) (2009) 498–511, [http://dx.doi.org/10.1109/TNN.2008.2010350](https://arxiv.org/abs/10.1109/TNN.2008.2010350).
- [7] U. Alon, E. Yahav, On the bottleneck of graph neural networks and its practical implications, in: 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3–7, 2021, OpenReview.net, 2021.
- [8] C. Morris, M. Ritzert, M. Fey, W.L. Hamilton, J.E. Lenssen, G. Rattan, M. Grohe, Weisfeiler and Leman go neural: higher-order graph neural networks, AAAI 33 (01) (2019) 4602–4609, [http://dx.doi.org/10.1609/aaai.v33i01.33014602](https://arxiv.org/abs/10.1609/aaai.v33i01.33014602).
- [9] E. Strubell, A. Ganesh, A. McCallum, Energy and policy considerations for deep learning in NLP, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Florence, Italy, 2019, pp. 3645–3650, [http://dx.doi.org/10.18653/v1/P19-1355](https://arxiv.org/abs/10.18653/v1/P19-1355).
- [10] N.C. Thompson, K. Greenewald, K. Lee, G.F. Manso, The computational limits of deep learning, 2022, [http://dx.doi.org/10.48550/arXiv.2007.05558](https://arxiv.org/abs/10.48550/arXiv.2007.05558), [arXiv:2007.05558](https://arxiv.org/abs/2007.05558) [cs, stat].
- [11] R. Schwartz, J. Dodge, N.A. Smith, O. Etzioni, Green AI, Commun. ACM 63 (12) (2020) 54–63, [http://dx.doi.org/10.1145/3381831](https://arxiv.org/abs/10.1145/3381831).
- [12] I. Chami, S. Abu-El-Haija, B. Perozzi, C. Ré, K. Murphy, Machine learning on graphs: a model and comprehensive taxonomy, J. Mach. Learn. Res. 23 (89) (2022) 1–64, URL [http://jmlr.org/papers/v23/20-852.html](https://arxiv.org/abs/10.1109/TNNLS.2020.2978386).
- [13] Z. Yang, G. Zhang, J. Wu, J. Yang, Q.Z. Sheng, S. Xue, C. Zhou, C. Aggarwal, H. Peng, W. Hu, E. Hancock, P. Liò, State of the art and potentialities of graph-level learning, 2023, [http://dx.doi.org/10.48550/arXiv.2301.05860](https://arxiv.org/abs/10.48550/arXiv.2301.05860), [arXiv:2301.05860](https://arxiv.org/abs/2301.05860) [cs].
- [14] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, P.S. Yu, A comprehensive survey on graph neural networks, IEEE Trans. Neural Netw. Learn. Syst. 32 (1) (2021) 4–24, [http://dx.doi.org/10.1109/TNNLS.2020.2978386](https://arxiv.org/abs/10.1109/TNNLS.2020.2978386).
- [15] M.M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, P. Vandergheynst, Geometric deep learning: Going beyond Euclidean data, IEEE Signal Process. Mag. 34 (4) (2017) 18–42, [http://dx.doi.org/10.1109/MSP.2017.2693418](https://arxiv.org/abs/10.1109/MSP.2017.2693418).
- [16] A.K. Debnath, R.L. Lopez de Compadre, G. Debnath, A.J. Shusterman, C. Hansch, Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro

- compounds. Correlation with molecular orbital energies and hydrophobicity, *J. Med. Chem.* 34 (2) (1991) 786–797, <http://dx.doi.org/10.1021/jm00106a046>.
- [17] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, J. Leskovec, Open graph benchmark: datasets for machine learning on graphs, 2021, [arXiv:2005.00687](https://arxiv.org/abs/2005.00687) [cs, stat].
- [18] N. Wale, I.A. Watson, G. Karypis, Comparison of descriptor spaces for chemical compound retrieval and classification, *Knowl. Inf. Syst.* 14 (3) (2008) 347–375, <http://dx.doi.org/10.1007/s10115-007-0103-5>.
- [19] K.M. Borgwardt, C.S. Ong, S. Schonauer, S.V.N. Vishwanathan, A.J. Smola, H.-P. Kriegel, Protein function prediction via graph kernels, *Bioinformatics* 21 (Suppl 1) (2005) i47–i56, <http://dx.doi.org/10.1093/bioinformatics/bti1007>.
- [20] W.L. Hamilton, Graph representation learning, in: *Synthesis Lectures on Artificial Intelligence and Machine Learning*, Springer International Publishing, Cham, 2020.
- [21] I. Aleksander, M. De Gregorio, F.M.G. França, P.M.V. Lima, H. Morton, A brief introduction to weightless neural systems, in: 17th European Symposium on Artificial Neural Networks, ESANN 2009, Bruges, Belgium, April 22–24, 2009, 2009.
- [22] I.D. Miranda, A. Arora, Z. Susskind, L.A. Villon, R.F. Katopodis, D.L. Dutra, L.S. De Araújo, P.M. Lima, F.M. França, L.K. John, M. Breternitz, LogicWiSARD: memoryless synthesis of weightless neural networks, in: 2022 IEEE 33rd International Conference on Application-Specific Systems, Architectures and Processors, ASAP, IEEE, Gothenburg, Sweden, 2022, pp. 19–26, <http://dx.doi.org/10.1109/ASAP54787.2022.00014>.
- [23] L.A. Villon, Z. Susskind, A.T. Bacellar, I.D. Miranda, L.S. De Araújo, P.M. Lima, M. Breternitz, L.K. John, F.M. França, D.L. Dutra, A conditional branch predictor based on weightless neural networks, *Neurocomputing* 555 (2023) 126637, <http://dx.doi.org/10.1016/j.neucom.2023.126637>.
- [24] A.S.L. Filho, G.P. Guarisa, L.A.D.L. Filho, L.F.R. Oliveira, F.M.G. França, P.M.V. Lima, Wisardpkg – a library for WiSARD-based models, 2020, [arXiv:2005.00887](https://arxiv.org/abs/2005.00887) [cs].
- [25] F.M.G. França, M. De Gregorio, P.M.V. Lima, W.R. Oliveira, Advances in weightless neural systems, in: *European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning*, 2014, pp. 497–504.
- [26] A.L. Filho, G.P. Guarisa, L.L. Filho, L.F.R. Oliveira, C.A.N. Cosenza, F.M.G. França, P.M.V. Lima, Interpretation of model agnostic classifiers via local mental images, in: 28th European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning, ESANN 2020, Bruges, Belgium, October 2–4, 2020, 2020, pp. 37–42.
- [27] Z. Susskind, A. Arora, I.D.S. Miranda, L.A.Q. Villon, R.F. Katopodis, L.S. De Araújo, D.L.C. Dutra, P.M.V. Lima, F.M.G. França, M. Breternitz, L.K. John, Weightless neural networks for efficient edge inference, in: *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, ACM, Chicago Illinois, 2022, pp. 279–290, <http://dx.doi.org/10.1145/3559009.3569680>.
- [28] D.F.P. Souza, F.M.G. França, P.M.V. Lima, Spatio-temporal pattern classification with KernelCanvas and WiSARD, in: 2014 Brazilian Conference on Intelligent Systems, IEEE, Sao Paulo, Brazil, 2014, pp. 228–233, <http://dx.doi.org/10.1109/BRACIS.2014.49>.
- [29] P. Boldi, S. Vigna, Axioms for centrality, *Internet Math.* 10 (2013) 222–262.
- [30] L. Page, S. Brin, R. Motwani, T. Winograd, The PageRank citation ranking: Bringing order to the web, in: *The Web Conference*, 1999.
- [31] B. Vargas, Exploring Pagerank Algorithms: Power Iteration & Monte Carlo Methods (Ph.D. thesis), California State University, San Marcos, 2020.
- [32] L. Hébert-Dufresne, J.A. Grochow, A. Allard, Multi-scale structure and topological anomaly detection via a new network statistic: The onion decomposition, *Sci. Rep.* 6 (1) (2016) 31708, <http://dx.doi.org/10.1038/srep31708>.
- [33] Z. Wan, Y. Mahajan, B.W. Kang, T.J. Moore, J.-H. Cho, A survey on centrality metrics and their network resilience analysis, *IEEE Access* 9 (2021) 104773–104819, <http://dx.doi.org/10.1109/ACCESS.2021.3094196>.
- [34] A. Kamiński, P. Cholda, A. Jajszczyk, Assessing the structural complexity of computer and communication networks, *ACM Comput. Surv.* 47 (4) (2015) 1–36, <http://dx.doi.org/10.1145/2755621>.
- [35] M. Hearst, S. Dumais, E. Osuna, J. Platt, B. Scholkopf, Support vector machines, *IEEE Intell. Syst. Appl.* 13 (4) (1998) 18–28, <http://dx.doi.org/10.1109/5254.708428>.
- [36] N.M. Kriege, F.D. Johansson, C. Morris, A survey on graph kernels, *Appl. Netw. Sci.* 5 (1) (2020) 6, <http://dx.doi.org/10.1007/s41109-019-0195-3>.
- [37] N. Shervashidze, P. Schweitzer, E.J. van Leeuwen, K. Mehlhorn, K.M. Borgwardt, Weisfeiler–Lehman graph kernels, *J. Mach. Learn. Res.* 12 (null) (2011) 2539–2561.
- [38] M. Sugiyama, K. Borgwardt, Halting in random walk kernels, in: C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, R. Garnett (Eds.), in: *Advances in Neural Information Processing Systems*, vol. 28, Curran Associates, Inc., 2015.
- [39] G. Nikolentzos, G. Siglidis, M. Vazirgiannis, Graph kernels: A survey, *Jair* 72 (2021) 943–1027, <http://dx.doi.org/10.1613/jair.1.13225>, URL <http://jair.org/index.php/jair/article/view/13225>.
- [40] A.J. Izenman, Introduction to manifold learning, *WIREs Comput. Stats* 4 (5) (2012) 439–446, <http://dx.doi.org/10.1002/wics.1222>.
- [41] U. Von Luxburg, A tutorial on spectral clustering, *Stat. Comput.* 17 (4) (2007) 395–416, <http://dx.doi.org/10.1007/s11222-007-9033-z>.
- [42] H. Cai, V.W. Zheng, K.C.-C. Chang, A comprehensive survey of graph embedding: problems, techniques, and applications, *IEEE Trans. Knowl. Data Eng.* 30 (9) (2018) 1616–1637, <http://dx.doi.org/10.1109/TKDE.2018.2807452>.
- [43] P. Goyal, E. Ferrara, Graph embedding techniques, applications, and performance: A survey, *Knowl.-Based Syst.* 151 (2018) 78–94, <http://dx.doi.org/10.1016/j.knsys.2018.03.022>.
- [44] N. Navarin, D.V. Tran, A. Sperduti, Universal readout for graph convolutional neural networks, in: 2019 International Joint Conference on Neural Networks, IJCNN, IEEE, Budapest, Hungary, 2019, pp. 1–7, <http://dx.doi.org/10.1109/IJCNN.2019.8852103>.
- [45] B. Perozzi, R. Al-Rfou, S. Skiena, DeepWalk: online learning of social representations, in: *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, New York New York USA, 2014, pp. 701–710, <http://dx.doi.org/10.1145/2623330.2623732>.
- [46] A. Grover, J. Leskovec, Node2vec: Scalable feature learning for networks, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, San Francisco California USA, 2016, pp. 855–864, <http://dx.doi.org/10.1145/2939672.2939754>.
- [47] L.F. Ribeiro, P.H. Saverese, D.R. Figueiredo, Struc2vec: Learning node representations from structural identity, in: *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, Halifax NS Canada, 2017, pp. 385–394, <http://dx.doi.org/10.1145/3097983.3098061>.
- [48] K.M. Khaki, Weightless Neural Networks for Face Recognition (Ph.D. thesis), Brunel University School of Engineering and Design, 2013.
- [49] W.W. Bledsoe, I. Browning, Pattern recognition and reading by machine, in: *Proceedings of Eastern Joint Computer Conference*, 1959, pp. 225–232.
- [50] W.S. McCulloch, W. Pitts, A logical calculus of the ideas immanent in nervous activity, *Bull. Math. Biophys.* 5 (1943) 115–133.
- [51] B.P. Grieco, P.M. Lima, M. De Gregorio, F.M. França, Producing pattern examples from “mental” images, *Neurocomputing* 73 (7–9) (2010) 1057–1064, <http://dx.doi.org/10.1016/j.neucom.2009.11.015>.
- [52] G. Coda, M. De Gregorio, A. Sorgente, P. Vanacore, Improving the DRASiW performance by exploiting its own “Mental Images”, in: *ESANN 2023 Proceedings*, Ciaco - i6doc.com, Bruges (Belgium) and online, 2023, pp. 363–368, <http://dx.doi.org/10.14428/esann/2023.ES2023-25>.
- [53] D.O. Cardoso, D.S. Carvalho, D.S. Alves, D.F. Souza, H.C. Carneiro, C.E. Pedreira, P.M. Lima, F.M. França, Financial credit analysis via a clustering weightless neural classifier, *Neurocomputing* 183 (2016) 70–78, <http://dx.doi.org/10.1016/j.neucom.2015.06.105>.
- [54] I. Wickert, F.M.G. França, AUTOWISARD: unsupervised modes for the wisard, in: *Connectionist Models of Neurons, Learning Processes, and Artificial Intelligence*, vol. 2084, Springer Berlin Heidelberg, Berlin, Heidelberg, 2001, pp. 435–441, http://dx.doi.org/10.1007/3-540-45720-8_51.
- [55] L.A. Lusquino Filho, L.F. Oliveira, A.L. Filho, G.P. Guarisa, L.M. Felix, P.M. Lima, F.M. França, Extending the weightless WiSARD classifier for regression, *Neurocomputing* 416 (2020) 280–291, <http://dx.doi.org/10.1016/j.neucom.2019.12.134>.
- [56] R.F. Katopodis, P.M. Lima, F.M. França, Functional gradient descent for n -tuple regression, *Neurocomputing* 500 (2022) 1016–1028, <http://dx.doi.org/10.1016/j.neucom.2022.05.114>.
- [57] L. Santiago, L. Verona, F. Rangel, F. Firmino, D.S. Menasché, W. Caarls, M. Breternitz Jr., S. Kundu, P.M. Lima, F.M. França, Weightless neural networks as memory segmented bloom filters, *Neurocomputing* 416 (2020) 292–304, <http://dx.doi.org/10.1016/j.neucom.2020.01.115>.
- [58] O.O. Napoli, A.M. De Almeida, E. Borin, M. Breternitz, Memory-efficient DRASiW Models, *Neurocomputing* 610 (2024) 128443, <http://dx.doi.org/10.1016/j.neucom.2024.128443>.
- [59] L.A. Lusquino Filho, F.M.G. França, P.M.V. Lima, WiSARD-based ensemble learning, in: *ESANN 2023 Proceedings*, Ciaco - i6doc.com, Bruges (Belgium) and online, 2023, pp. 387–392, <http://dx.doi.org/10.14428/esann/2023.ES2023-76>.
- [60] Z. Susskind, A. Arora, I.D.S. Miranda, A.T.L. Bacellar, L.A.Q. Villon, R.F. Katopodis, L.S. de Araújo, D.L.C. Dutra, P.M.V. Lima, F.M.G. França, M. Breternitz Jr., L.K. John, ULEEN: A novel architecture for ultra-low-energy edge neural networks, *ACM Trans. Arch. Code Optim.* 20 (4) (2023) 61:1–61:24, <http://dx.doi.org/10.1145/3629522>.
- [61] A.T.L. Bacellar, Z. Susskind, M. Breternitz Jr., E. John, L.K. John, P.M.V. Lima, F.M. França, Differentiable weightless neural networks, in: R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, F. Berkenkamp (Eds.), *Proceedings of the 41st International Conference on Machine Learning*, in: *Proceedings of Machine Learning Research*, vol. 235, PMLR, 2024, pp. 2277–2295.
- [62] A. Kappaun, K. Camargo, F. Rangel, F. Firmino, P.M.V. Lima, J. Oliveira, Evaluating binary encoding techniques for WiSARD, in: 2016 5th Brazilian Conference on Intelligent Systems, BRACIS, IEEE, Recife, Brazil, 2016, pp. 103–108, <http://dx.doi.org/10.1109/BRACIS.2016.029>.
- [63] R. Barbosa, D.O. Cardoso, D. Carvalho, F.M. França, Weightless neuro-symbolic GPS trajectory classification, *Neurocomputing* 298 (2018) 100–108, <http://dx.doi.org/10.1016/j.neucom.2017.11.075>.

- [64] A.T.L. Bacellar, Z. Susskind, L.A.Q. Villon, I.D.S. Miranda, L.S. De Araújo, D.L. Cadette Dutra, M. Breternitz Jr., L. John, P. Lima, F. França, Distributive thermometer: a new unary encoding for weightless neural networks, in: ESANN 2022 Proceedings, Ciaco - i6doc.com, Bruges (Belgium) and online event, 2022, pp. 31–36, <http://dx.doi.org/10.14428/esann/2022.ES2022-94>.
- [65] H.C.C. Carneiro, F.M.G. França, P.M.V. Lima, Multilingual part-of-speech tagging with weightless neural networks, *Neural Netw.* 66 (2015) 11–21, <http://dx.doi.org/10.1016/j.neunet.2015.02.012>.
- [66] S. Garcia, J. Luengo, J.A. Sáez, V. López, F. Herrera, A survey of discretization techniques: taxonomy and empirical analysis in supervised learning, *IEEE Trans. Knowl. Data Eng.* 25 (4) (2013) 734–750, <http://dx.doi.org/10.1109/TKDE.2012.35>.
- [67] P. Xavier, M. De Gregorio, F.M.G. França, P.M.V. Lima, Detection of elementary particles with the WiSARD n -tuple classifier, in: 28th European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning, ESANN 2020, Bruges, Belgium, October 2–4, 2020, 2020, pp. 643–648.
- [68] M. Berlingiero, D. Koutra, T. Eliassi-Rad, C. Faloutsos, NetSimile: A scalable approach to size-independent network similarity, 2012, [arXiv:1209.2684](https://arxiv.org/abs/1209.2684) [physics, stat].
- [69] M. Berlingiero, D. Koutra, T. Eliassi-Rad, C. Faloutsos, Network similarity via multiple social theories, in: Proceedings of the 2013 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining, ACM, Niagara Ontario Canada, 2013, pp. 1439–1440, <http://dx.doi.org/10.1145/2492517.2492582>.
- [70] N. de Lara, E. Pineau, A simple baseline algorithm for graph classification, 2018, [arXiv:1810.09155](https://arxiv.org/abs/1810.09155) [cs, stat].
- [71] A. Tsitsulin, D. Mottin, P. Karras, A. Bronstein, E. Müller, NetLSD: hearing the shape of a graph, in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2018, pp. 2347–2356, <http://dx.doi.org/10.1145/3219819.3219991>, [arXiv:1805.10712](https://arxiv.org/abs/1805.10712).
- [72] A. Tsitsulin, M. Munkhoeva, B. Perozzi, Just SLAQ when you approximate: accurate spectral distances for web-scale graphs, in: Proceedings of The Web Conference 2020, 2020, pp. 2697–2703, <http://dx.doi.org/10.1145/3366423.3380026>, [arXiv:2003.01282](https://arxiv.org/abs/2003.01282).
- [73] S. Verma, Z.-L. Zhang, Hunt for the unique, stable, sparse and fast feature learning on graphs, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS '17, Curran Associates Inc., 2017, pp. 87–97.
- [74] B. Rozemberczki, R. Sarkar, Characteristic functions on graphs: birds of a feather, from statistical descriptors to parametric models, 2020, [arXiv:2005.07959](https://arxiv.org/abs/2005.07959) [cs, stat].
- [75] L. Wang, C. Huang, W. Ma, X. Cao, S. Vosoughi, Graph embedding via diffusion-wavelets-based node feature distribution characterization, in: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, 2021, pp. 3478–3482, <http://dx.doi.org/10.1145/3459637.3482115>.
- [76] G. Li, M. Semerci, B. Yener, M.J. Zaki, Effective graph classification based on topological and label attributes, *Stat. Anal. Data Min.* 5 (4) (2012) 265–283, <http://dx.doi.org/10.1002/sam.11153>.
- [77] K.S. Bohnsack, M. Kaden, J. Voigt, T. Villmann, Efficient classification learning of biochemical structured data by means of relevance weighting for sensoric response features, in: ESANN 2022 Proceedings, Ciaco - i6doc.com, Bruges (Belgium) and online event, 2022, pp. 445–450, <http://dx.doi.org/10.14428/esann/2022.ES2022-36>.
- [78] M. Nagy, R. Molontay, Network classification based structural analysis of real networks and their model-generated counterparts, 2022, [arXiv:1810.08498](https://arxiv.org/abs/1810.08498) [physics].
- [79] C. Cai, Y. Wang, A simple yet effective baseline for non-attributed graph classification, 2022, [arXiv:1811.03508](https://arxiv.org/abs/1811.03508) [cs, stat].
- [80] S. Aliakbari, J. Habibi, A. Movaghar, Quantification and comparison of degree distributions in complex networks, in: 7th International Symposium on Telecommunications, IST'2014, IEEE, 2014, pp. 464–469, <http://dx.doi.org/10.1109/ISTEL.2014.7000748>.
- [81] R.G. Barbastefano, D.M. de Araujo Carvalho, M.C. Lippi, Process mining classification with a weightless neural network, in: A.M.T. Thomé, R.G. Barbastefano, L.F. Scavarda, J.C.G. dos Reis, M.P.C. Amorim (Eds.), *Industrial Engineering and Operations Management*, Springer International Publishing, Cham, 2020, pp. 349–356, http://dx.doi.org/10.1007/978-3-030-56920-4_28.
- [82] R. Barbosa, D. Carvalho, P.M.V. Lima, F.M.G. França, A bag of nodes primer on weightless graph classification, in: ESANN 2021 Proceedings, 2021, <http://dx.doi.org/10.14428/esann/2021.ES2021-107>.
- [83] F.N. David, N.L. Johnson, The probability integral transformation when parameters are estimated from the sample, *Biometrika* 35 (1/2) (1948) 182–190, <http://dx.doi.org/10.2307/2332638>.
- [84] C. Morris, N.M. Kriege, F. Bause, K. Kersting, P. Mutzel, M. Neumann, TUDataset: A collection of benchmark datasets for learning with graphs, in: *ICML 2020 Workshop on Graph Representation Learning and beyond*, GRL+ 2020, 2020.
- [85] P. Yanardag, S. Vishwanathan, Deep graph kernels, in: Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, Sydney NSW Australia, 2015, pp. 1365–1374, <http://dx.doi.org/10.1145/2783258.2783417>.
- [86] I. Schomburg, BRENDA, the enzyme database: updates and major new developments, *Nucleic Acids Res.* 32 (90001) (2004) 431D–433, <http://dx.doi.org/10.1093/nar/gkh081>.
- [87] N. Kriege, P. Mutzel, Subgraph matching kernels for attributed graphs, 2012, [arXiv:1206.6483](https://arxiv.org/abs/1206.6483) [cs, stat].
- [88] S. Kim, J. Chen, T. Cheng, A. Gindulyte, J. He, S. He, Q. Li, B.A. Shoemaker, P.A. Thiessen, B. Yu, L. Zaslavsky, J. Zhang, E.E. Bolton, PubChem 2023 update, *Nucleic Acids Res.* 51 (D1) (2023) D1373–D1380, <http://dx.doi.org/10.1093/nar/gkac956>.
- [89] P.D. Dobson, A.J. Doig, Distinguishing enzyme structures from non-enzymes without alignments, *J. Mol. Biol.* 330 (4) (2003) 771–783, [http://dx.doi.org/10.1016/S0022-2836\(03\)00628-4](http://dx.doi.org/10.1016/S0022-2836(03)00628-4).
- [90] G. Nikolentzos, M. Chatzianastasis, M. Vazirgiannis, Weisfeiler and leman go hyperbolic: Learning distance preserving node representations, in: F. Ruiz, J. Dy, J.-W. van de Meent (Eds.), *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, in: *Proceedings of Machine Learning Research*, vol. 206, PMLR, 2023, pp. 1037–1054.
- [91] C. Yang, Q. Wu, J. Wang, J. Yan, Graph neural networks are inherently good generalizers: insights by bridging gnns and mlps, in: *The Eleventh International Conference on Learning Representations*, 2023, URL <https://openreview.net/forum?id=dqnNW2omZL6>.
- [92] F. Errica, M. Podda, D. Bacciu, A. Micheli, A fair comparison of graph neural networks for graph classification, 2022, [arXiv:1912.09893](https://arxiv.org/abs/1912.09893) [cs.LG].
- [93] M. Fey, J.E. Lenssen, Fast graph representation learning with PyTorch Geometric, in: *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [94] P. Welke, T.H. Schulz, On the necessity of graph kernel baselines, in: *Graph Embedding and Mining Workshop at ECML PKDD*, 2019.
- [95] R. Liu, S. Cantürk, F. Wenkel, S. McGuire, X. Wang, A. Little, M. Bray, B. Rieck, M. Hirn, G. Wolf, L. Rampásek, Taxonomy of benchmarks in graph representation learning, in: B. Rieck, R. Pascanu (Eds.), *Proceedings of the First Learning on Graphs Conference*, in: *Proceedings of Machine Learning Research*, vol. 198, PMLR, 2022, pp. 6:1–6:25.



Raul B. Barbosa is a Ph.D. candidate in Computer and Systems Engineering at Federal University of Rio de Janeiro (UFRJ), Brazil. He received his B.Sc. in Computer Science and M.Sc. in Computer and Systems Engineering in 2014 and 2018, respectively, all from Federal University of Rio de Janeiro (UFRJ). His research interests include geospatial and graph data mining, and weightless neural networks.



Diego L.C. Dutra is a Professor at the Federal University of Rio de Janeiro (UFRJ), Brazil, leading the HEADS research group and contributing to the COMPASS Laboratory. With a D.Sc. in Systems Engineering and Computer Science from UFRJ, and postdoctoral experience at UFRJ and Aalto University, his research focuses on computer architecture, high-performance computing, cloud and mobile systems, operating systems, and Software-Defined Systems.



Professor Priscila Machado Vieira Lima holds an undergraduate degree in Mathematics/Informatics (1981, Magna cum Laude) from IM/UFRJ, M. Sc. (1987) in Computer Science from COPPE/UFRJ and Ph. D (2000) in Computer Science from Imperial College of Science Technology and Medicine. She is currently faculty at the Tércio Pacitti Institute (NCE) of UFRJ and at the Systems Engineering and Computer Science Program, COPPE, UFRJ, having occupied the Artificial Intelligence Cathedra of the High Studies Brazilian School of UFRJ from 2021 to 2022. She was also the head of the graduate programmes PPGMMC/UFRJ (2012–2013), which proposal she also conducted, and PPGI/UFRJ (2015–2016). She has published around 120 scientific papers and successfully supervised over 40 graduate students, who are in industry, government and academic positions, in Brazil and abroad. Her experience in Computer Science focus on subjects related to Artificial Intelligence, among which are the topics: neuro-symbolic reasoning, weightless neural systems, weightless neural circuits, integration AI-Optimization and ethical guidelines for AI.



Diego Carvalho (M'98-SM'19) was born in Rio de Janeiro, Brazil in 1970. He received his B.S. in Production Engineering from UFRJ and his M.Sc. and D.Sc. in Systems Engineering and Computer Science from PESC/COPPE. From 1993 to 1996, he was a Computer Research Assistant with the DELPHI Experiment at CERN. From 1997 to 2011, he was amongst the leading researchers of various EU-funded grid computing projects. Since 2006, he has been a professor at the Department of Production Engineering of CEFET/RJ, and his research interests include distributed systems, network engineering, parallel architectures, grid technologies, data mining, and big data. Dr. Carvalho is a member of the Brazilian Association of Production Engineering, the Brazilian Society for the Advancement of Science, and a senior member of IEEE.



Felipe Maia Galvão França Electronics Engineer from Federal University of Rio de Janeiro (UFRJ) (1982), M.Sc. in Computer Science from UFRJ (1987) and Ph.D. from Imperial College of Science Technology and Medicine (1994). He is now a Visiting Researcher at Google LLC, Sunnyvale, CA. During the development of this work he was a Visiting Researcher at Instituto de Telecomunicações — Universidade do Porto (UPorto), and he is a Retired Full Professor at the Systems Engineering and Computer Science Program, COPPE, UFRJ. He has published nearly 300 scientific papers and 2 granted patents. He has experience in Computer Science and Electronics Engineering, acting on the following subjects: artificial neural networks, weightless neural networks, computer architecture, cryptographic circuits, dataflow computing, distributed algorithms, collective robotics, intelligent transportation systems. Up to this date, he has successfully supervised 33 Ph.D. students and 67 M.Sc. students, who are all in industry, government and academic positions, in Brazil, Canada, China, Germany, Japan, Portugal, USA and the UK.